南京大學

研 究 生 毕 业 论 文
（申 请 硕 士 学 位）

论 文 题 目 面向广告与商业化内容的管理、应用与治理平台的设计与实现

作 者 姓 名 靳琦清

院 系 软件学院

专 业 名 称 软件工程

指 导 教 师 刘峰 高级工程师

2026 年 2 月 27 日

学　　号：**522024320076**

论文答辩日期：**xxxx** 年 **xx** 月 **xx** 日

指 导 教 师：　　（签字）

Design and Implementation of a Platform for the Management, Application, and Governance of Advertising and Commercial Content

by

QiQing Jin

Supervised by

Feng Liu, Senior Engineer

A dissertation submitted to
the graduate school of Nanjing University
in partial fulfilment of the requirements for the degree of
MASTER
in
Software Engineering

Software Institute
Nanjing University

February 27, 2026

南京大学研究生毕业论文中文摘要首页用纸

毕业论文题目：面向广告与商业化内容的管理、应用与治理平台的设计与实现

软件工程 专业 2024 级硕士生姓名：靳琦清

指导教师（姓名、职称）：刘峰 高级工程师

摘　要

随着互联网广告与商业化业务的快速发展，广告产品形态、业务规则及运营策略日益复杂，围绕广告投放、产品说明、业务规范与运营实践等内容不断沉淀，形成了规模庞大、类型多样的专业内容资产。这类内容在业务人员培训、运营决策支持以及客户服务等场景中发挥着重要作用。然而，随着业务线数量和内容规模的持续增长，现有内容平台在内容统一管理、高效应用以及质量保障等方面逐渐暴露出诸如内容分散、复用困难、维护成本高以及质量难以评估和治理等问题，已难以满足广告与商业化场景下的实际需求。

针对上述问题，本文在分析现有内容管理与知识平台相关工作的基础上，结合企业内部广告与商业化业务的实际应用场景，设计并实现了一套面向广告与商业化内容的管理、应用与治理平台。平台以内容全生命周期为核心，对内容的建模、组织、应用与治理进行系统化设计，旨在提升内容资产的统一管理能力、应用效果以及质量可控性。

在内容管理方面，平台提出了一种基于空间与应用分层的内容组织模型，实现对不同广告业务线及其应用场景的隔离与统一管理，并支持文章、问答、课程和案例等多种内容形态的规范化管理。结合类目树、标签体系、权限控制与审批流程，平台有效降低了内容维护成本，提升了内容资产的复用性与一致性。在内容应用方面，平台在支持传统页面化内容展示的基础上，引入基于检索增强生成（Retrieval-Augmented Generation，RAG）的智能问答能力，通过内容切片、向量索引、多通道召回与重排等技术手段，为上游智能应用提供高相关性的内容支持，拓展了内容资产的应用方式。在内容治理方面，针对内容规模扩大带来的质量问题，本文构建了一种基于重复度、歧义度与覆盖度的内容治理机制，结合向量检索与大语言模型实现对内容质量的自动检测，并通过线上化治理任务实现内容治理的闭环处理。此外，平台通过数据洞察模块对内

容使用效果和治理结果进行统计分析，为内容优化与治理策略调整提供数据支持。

实际应用结果表明，本文所设计和实现的平台在广告与商业化业务场景中有效提升了内容管理效率与应用效果，降低了内容治理的人力成本，对面向广告与商业化内容的内容平台建设具有一定的工程实践价值和参考意义。

关键词： 广告与商业化内容；内容管理平台；检索增强生成（RAG）；内容治理；智能问答

南京大学研究生毕业论文英文摘要首页用纸

THESIS: Design and Implementation of a Platform for the Management, Application, and Governance of Advertising and Commercial Content

SPECIALIZATION: Software Engineering

POSTGRADUATE: QiQing Jin

MENTOR: Feng Liu, Senior Engineer

Abstract

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

keywords: keyword1 keyword2

目　　录

中文摘要 ····· **i**
英文摘要 ····· **iii**
目　　录 ····· **v**
插图清单 ····· **ix**
附表清单 ····· **xi**
第一章　引言 ····· **1**
1.1　项目背景 ····· 1
1.2　国内外发展现状及分析 ····· 2
1.3　本文主要工作 ····· 3
1.4　论文的组织结构 ····· 4
第二章　相关技术综述 ····· **7**
2.1　远程过程调用 RPC ····· 7
2.2　文档数据库 MongoDB ····· 8
2.3　消息队列 RocketMQ ····· 8
2.4　全文搜索引擎 ElasticSearch ····· 10
2.5　Embedding 与向量检索技术 ····· 11
2.6　检索增强生成 RAG ····· 12
2.7　本章小结 ····· 13
第三章　内容管理、应用与治理平台的分析与设计 ····· **15**
3.1　系统整体概述 ····· 15
3.2　内容管理、应用与治理平台需求分析 ····· 17
3.2.1　用户角色 ····· 17
3.2.2　内容管理功能需求 ····· 18
3.2.3　内容应用功能需求 ····· 28
3.2.4　内容治理功能需求 ····· 31
3.2.5　数据洞察功能需求 ····· 34
3.2.6　系统非功能需求 ····· 37

3.3 系统整体设计 ····· 37
3.4 系统的模块设计 ····· 40
3.4.1 内容管理模块设计 ····· 40
3.4.2 内容应用模块设计 ····· 42
3.4.3 内容治理模块设计 ····· 45
3.4.4 数据洞察模块设计 ····· 47
3.5 数据库设计 ····· 48
3.5.1 概念结构设计 ····· 49
3.5.2 表结构设计 ····· 49
3.6 本章小结 ····· 58
第四章 内容管理、应用与治理平台的实现 ····· 59
4.1 内容管理模块的实现 ····· 59
4.1.1 多形态内容统一建模与管理 ····· 60
4.1.2 内容组织与引用机制 ····· 64
4.1.3 权限管理与审批流程 ····· 65
4.2 内容应用模块的实现 ····· 67
4.2.1 页面化内容应用机制 ····· 67
4.2.2 多形态内容前置处理与索引构建 ····· 67
4.2.3 基于 RAG 的多通道召回与重排 ····· 68
4.2.4 生成式问答流程实现 ····· 68
4.3 内容治理模块的实现 ····· 70
4.3.1 重复度、歧义度与覆盖度检测 ····· 70
4.3.2 治理任务线上化与闭环机制 ····· 70
4.4 内容治理模块的实现 ····· 72
4.4.1 重复度、歧义度与覆盖度检测 ····· 72
4.4.2 治理任务线上化与闭环机制 ····· 72
4.5 数据洞察模块的实现 ····· 72
4.5.1 内容应用数据采集机制 ····· 72
4.5.2 会话分析与效果指标设计 ····· 72
4.6 本章小结 ····· 72
第五章 内容管理、应用与治理平台测试 ····· 73
5.1 系统测试环境 ····· 73

5.2 单元测试 ····· 73
5.3 功能测试 ····· 73
5.4 本章小结 ····· 73
第六章 总结与展望 ····· 75
6.1 总结 ····· 75
6.2 展望 ····· 75
参考文献 ····· 77
A 附录示例 ····· 79
A.1 main 函数 ····· 79
致　　谢 ····· 81
版权及论文原创性说明 ····· 83
《学位论文出版授权书》 ····· 85

插图清单

2-1 RocketMQ 架构图 ····· 9

3-1 内容管理用例图 ····· 18
3-2 内容应用用例图 ····· 29
3-3 内容治理用例图 ····· 32
3-4 数据洞察用例图 ····· 34
3-5 系统架构图 ····· 38
3-6 系统逻辑视图 ····· 39
3-7 开发视图 ····· 39
3-8 物理视图 ····· 40
3-9 空间与应用管理时序图 ····· 41
3-10 内容创建与版本管理时序图 ····· 41
3-11 内容状态机图 ····· 41
3-12 内容权限鉴权流程图 ····· 42
3-13 内容发布流程时序图 ····· 43
3-14 应用引用内容时序图 ····· 43
3-15 内容展示流程示意图 ····· 44
3-16 内容搜索链路示意图 ····· 44
3-17 索引构建流程示意图 ····· 45
3-18 内容召回与重排流程示意图 ····· 45
3-19 重复度/歧义度模型检测流程示意图 ····· 46
3-20 质量治理任务线上化流程示意图 ····· 47
3-21 治理修订-发布-复检时序图 ····· 47
3-22 数据采集与指标生成链路示意图 ····· 48
3-23 数据洞察指标体系示意图 ····· 48
3-24 数据库 E-R 图 ····· 49

4-1 内容创建与版本管理关键代码 ····· 62

4-2 应用引用内容关键代码 …… 65
4-3 审批通过与发布事件触发关键代码 …… 66
4-4 页面化内容展示关键代码 …… 68
4-5 索引构建关键代码 …… 69
4-6 RAG 召回与重排关键代码 …… 71
4-7 生成式问答关键代码 …… 72

附表清单

3-1 用户角色表 …… 17
3-2 创建空间用例描述表 …… 19
3-3 创建应用用例描述表 …… 19
3-4 创建内容用例描述表 …… 20
3-5 编辑内容用例描述表 …… 21
3-6 删除内容用例描述表 …… 21
3-7 成员角色分配用例描述表 …… 22
3-8 权限策略配置用例描述表 …… 23
3-9 提交审核用例描述表 …… 24
3-10 内容审批用例描述表 …… 25
3-11 内容上下线用例描述表 …… 26
3-12 类目管理用例描述表 …… 27
3-13 标签管理用例描述表 …… 27
3-14 应用引用内容用例描述表 …… 28
3-15 内容展示用例描述表 …… 30
3-16 内容检索用例描述表 …… 30
3-17 生成式问答用例描述表 …… 31
3-18 质量检测用例描述表 …… 33
3-19 治理任务创建用例描述表 …… 33
3-20 治理任务处理用例描述表 …… 34
3-21 查看会话分析用例描述表 …… 35
3-22 查看内容效果用例描述表 …… 36
3-23 查看治理成效用例描述表 …… 36
3-24 RBAC 权限矩阵表 …… 42
3-25 空间表 …… 50
3-26 应用表 …… 50
3-27 内容表 …… 51

3-28 内容版本表 ····· 51
3-29 知识内容扩展表（knowledge_content） ····· 52
3-30 FAQ 内容扩展表（faq_content） ····· 52
3-31 课程内容扩展表（course_content） ····· 53
3-32 课节表（lesson） ····· 53
3-33 案例详情集合（MongoDB：case_detail） ····· 54
3-34 应用引用内容表 ····· 54
3-35 内容权限表（content_auth） ····· 55
3-36 审批表 ····· 55
3-37 文档表 ····· 56
3-38 切片表 ····· 56
3-39 内容-文档关联表 ····· 57
3-40 治理任务表 ····· 57

4-1 内容管理模块接口表 ····· 60
4-2 内容管理模块核心数据表 ····· 61

第一章　引言

1.1　项目背景

随着互联网广告与商业化业务的持续发展，广告产品形态、投放策略及业务规则不断演进，相关业务体系呈现出高度复杂化和专业化特征。在广告投放、产品能力说明、业务规范制定以及运营实践过程中，逐步沉淀了大量与广告与商业化业务密切相关的专业内容资产。这些内容涵盖业务规则解读、产品功能说明、操作流程指导以及典型业务案例等，是支撑广告与商业化业务运行和决策的重要基础。

随着业务规模扩大和业务线数量增加，广告与商业化内容在数量和类型上呈现持续增长趋势。内容逐步分散于不同业务线和系统中，缺乏统一的组织模型和管理规范，导致内容检索效率降低、复用成本上升，内容维护高度依赖人工经验，难以形成系统化、可持续的内容管理体系。

在内容规模不断扩大的同时，内容的应用方式也在持续演进。传统依赖人工检索和页面浏览的内容使用模式，已难以满足广告与商业化业务对效率和准确性的要求。随着检索增强生成和智能问答等技术在业务系统中的逐步应用，内容的使用方式正从“人工查找”向“智能获取”转变。这一变化对内容的结构化程度、可检索性以及一致性提出了更高要求，同时也显著提升了内容在业务决策和问题处理过程中的使用频率。

随着内容被更广泛、更高频地应用，内容质量问题逐渐凸显。一方面，部分内容在长期演进过程中形成重复和冗余，降低了内容体系的整体可用性；另一方面，不同内容在描述同一业务问题时可能存在表述不一致或语义歧义，在智能问答等自动化应用场景下容易被进一步放大，影响内容理解的准确性。此外，由于缺乏系统化分析和评估手段，内容体系在部分业务领域存在覆盖不足的问题。随着内容规模和应用深度的提升，传统依赖人工维护的方式已难以有效支撑内容质量的持续保障。

现有内容管理平台多以内容存储和页面展示为核心，主要提供基础的内容管理功能，在内容统一建模、多形态内容协同管理、智能化应用支持以及内容

质量治理等方面能力有限，难以满足广告与商业化场景下内容规模大、应用频繁且质量要求高的实际需求。基于上述背景，本文设计并实现了一套面向广告与商业化内容的管理、应用与治理平台，从内容全生命周期视角出发，对内容的生产、组织、应用和维护过程进行了系统性设计。平台在统一内容建模与组织方式的基础上，引入智能化的内容应用能力以及自动化与人工相结合的内容治理机制，实现了广告与商业化内容资产的统一管理、高效应用与质量可控，为复杂广告与商业化业务场景下的内容使用与持续维护提供了一种工程化解决方案。

1.2 国内外发展现状及分析

在内容管理与知识应用领域，国内外研究和工程实践主要围绕企业内容管理系统（Enterprise Content Management，ECM）和内容管理系统（Content Management System，CMS）展开。相关研究普遍关注内容的集中存储、统一建模、流程化管理以及权限控制等问题，通过引入内容生命周期管理和工作流机制，提高内容管理的规范性与可控性。在政务、教育及企业信息化场景中，此类系统已形成较为成熟的工程实践体系，在文档协同、权限隔离和合规管理方面发挥了重要作用。

在工程实践层面，国外以 Microsoft 推出的 SharePoint、Atlassian 的 Confluence 等平台为代表，提供了面向组织级内容的统一管理与协作能力，支持内容版本控制、权限配置以及结构化组织。这类平台在内容管理和协同办公方面具备较高成熟度，但其核心使用模式仍以人工检索和页面浏览为主，对内容的深度应用和智能化利用支持相对有限。国内企业也普遍建设了面向内部知识沉淀的内容管理或知识平台，用于支撑业务培训和经验复用，但整体上仍以管理和展示为主要目标。

随着业务复杂度和内容规模的持续提升，传统内容管理系统在内容应用层面的局限逐渐显现。近年来，随着大语言模型技术的发展，基于检索增强生成（Retrieval-Augmented Generation，RAG）的内容应用与智能问答技术成为研究热点。相关研究通过将外部知识检索与生成模型相结合，在一定程度上提升了知识密集型任务的准确性，并围绕检索策略、结果重排以及生成效果评估等方面展开了大量探索。部分研究进一步将 RAG 技术应用于多模态文档和垂直领域场景，引入图片、表格等非文本内容的检索与理解能力，用于提升复杂业务

问题的问答效果。

尽管 RAG 等智能内容应用技术在算法层面取得了显著进展，但现有研究多聚焦于模型能力和问答效果优化，通常假设知识来源具备较高质量和良好结构，较少关注内容在生产、维护和演进过程中的管理问题。在实际业务场景中，内容往往来源多样、形态复杂且质量参差不齐，单纯依赖智能检索和生成模型，难以从根本上保障内容应用的稳定性和可靠性。

与此同时，在内容质量与数据治理方向，已有研究从一致性、完整性、准确性以及治理流程等维度提出了较为系统的理论框架，强调通过指标化评估、流程化管理和工具化支撑提升内容与数据资产的可信度和可维护性。这类研究在数据治理和企业信息管理领域具有较成熟的理论基础。然而，在具体工程实践中，内容质量治理往往以离线分析或人工审查为主，与内容管理系统和智能应用系统相对独立，缺乏将质量检测结果直接反馈到内容管理和应用流程中的闭环机制，难以支撑内容规模持续增长和高频应用场景下的长期运营需求。

综合来看，现有研究和系统在内容管理、智能应用以及内容治理等方面均取得了一定成果，但整体上仍呈现出相对割裂的发展状态。传统内容管理系统侧重内容管理但缺乏智能化应用能力，智能问答和 RAG 相关研究强调内容应用效果却忽视内容管理和治理的工程复杂性，而内容质量治理研究又难以与实际内容应用形成闭环。在面向广告与商业化内容的复杂业务场景中，内容规模大、更新频繁且应用方式多样，单一侧重某一方面的解决方案难以满足实际需求。因此，如何在统一内容管理的基础上，引入智能化内容应用能力，并通过系统化的内容治理机制保障内容质量，实现管理、应用与治理协同演进的内容平台，仍有待进一步研究和工程实践。

1.3 本文主要工作

广告与商业化内容在实际业务中具有规模大、更新频繁、内容形态多样以及应用场景复杂等特点，传统内容平台在统一管理、高效应用和质量保障等方面逐渐暴露出不足。为解决上述问题，本文在分析国内外内容管理、智能内容应用以及内容治理相关研究和工程实践的基础上，结合企业内部广告与商业化业务的实际需求，设计并实现了一套面向广告与商业化内容的管理、应用与治理平台，对平台的整体架构与核心模块进行了系统性设计与实现。

本文所设计的平台以内容全生命周期管理为核心，对内容从创建、组织、

应用到治理的全过程进行统一支撑，减少内容在不同系统之间的割裂状态，提高内容资产的复用效率和质量可控性。围绕上述目标，本文的主要工作包括以下几个方面。

首先，在内容管理方面，设计并实现了一套统一的内容管理机制。平台通过引入空间与应用的分层组织模型，对不同广告业务线及其应用场景进行隔离与管理，支持文章、问答、课程和案例等多种内容形态的统一建模与维护。同时，结合类目树、标签体系、权限控制和审批流程，实现内容从创建、维护到上线的规范化管理，降低内容维护成本，提高内容资产的一致性和可复用性。

其次，在内容应用方面，平台在支持传统页面化内容展示的基础上，引入基于检索增强生成的智能问答能力，拓展内容资产的使用方式。针对不同内容形态的特点，设计了相应的内容切片、索引和召回策略，并通过多通道召回与重排机制提升内容匹配的相关性，为上游智能应用提供稳定、可靠的内容支撑，使内容能够在复杂业务场景中被高效获取和利用。

再次，在内容治理方面，针对内容规模扩大和应用频率提升带来的质量问题，构建了一套内容治理机制。平台从内容重复度、歧义度和覆盖度等维度对内容质量进行分析，通过向量检索和大语言模型判别实现内容质量的自动检测，并将检测结果转化为线上化治理任务，指派相关人员进行处理，从而形成内容治理的闭环流程，提升内容体系的整体质量和可维护性。

最后，在数据分析与反馈方面，平台通过数据洞察模块对内容的使用情况和治理效果进行统计分析，从页面访问、用户会话和内容使用等多个维度对内容应用效果进行量化评估，为内容优化和治理策略调整提供数据支持，进一步支撑平台的持续演进和优化。

综上所述，本文通过对广告与商业化内容管理、应用与治理平台的设计与实现，探索了一种将内容管理、智能应用与内容治理进行一体化整合的工程化解决方案，为复杂业务场景下内容资产的高效利用和长期维护提供了实践参考。

1.4 论文的组织结构

本文围绕面向广告与商业化内容的管理、应用与治理平台的设计与实现展开，全文共分为六章，各章节内容安排如下。

第一章为绪论，主要介绍了论文的研究背景和研究意义，分析了广告与商

业化内容在实际业务场景下面临的管理、应用和质量治理问题，并对国内外相关研究和工程实践的发展现状进行了综述，最后概述了本文的主要研究内容和工作安排。

第二章为相关技术综述，主要介绍了本文系统设计与实现过程中所涉及的关键技术和方法，包括内容管理相关技术、全文检索与向量检索技术、检索增强生成（RAG）技术以及大语言模型在智能问答中的应用等，为后续系统设计与实现提供技术基础。

第三章为平台需求分析与总体设计。该章节结合广告与商业化业务场景，对平台的功能需求和非功能需求进行了分析，在此基础上给出了平台的总体架构设计，并对各核心模块的设计思路进行了说明。

第四章为平台的详细设计与实现。该章节围绕内容管理、内容应用和内容治理等核心模块，对系统的关键功能设计和具体实现过程进行了详细阐述，并结合实际系统界面展示了平台的主要功能效果。

第五章为系统效果分析与评估。该章节通过对平台在实际业务场景中的运行情况进行分析，从内容管理效率、内容应用效果以及内容治理成效等多个维度对系统进行评估，验证平台设计的可行性和有效性。

第六章为总结与展望，对全文的研究工作进行了总结，分析了当前系统存在的不足，并对未来在功能扩展和技术优化等方面的研究方向进行了展望。

第二章　相关技术综述

2.1　远程过程调用 RPC

远程过程调用（Remote Procedure Call，RPC）是一种用于支持分布式系统中服务间通信的技术机制，其核心思想是通过统一的调用抽象，使得应用程序能够以类似本地函数调用的方式访问远程服务，从而屏蔽底层网络通信细节。通过 RPC，开发者无需直接处理消息传输、序列化以及连接管理等复杂问题，即可实现跨进程、跨节点的服务调用，这在现代分布式系统和微服务架构中具有广泛应用。

在 RPC 通信模型中，调用方通过接口定义与被调用方建立逻辑关联，调用请求在客户端侧被封装为消息并通过网络发送至服务端，服务端在完成对应逻辑处理后将结果返回给调用方。整个过程在调用语义上通常表现为同步调用，使得业务逻辑能够以顺序化的方式进行组织。通过这种方式，RPC 技术在保持调用语义清晰性的同时，实现了服务之间的解耦和功能拆分。

从系统实现角度看，RPC 框架通常围绕接口描述、数据序列化、通信协议以及调用调度等关键环节进行设计。接口描述用于定义服务的调用规范，序列化机制负责将结构化数据转换为可传输的二进制格式，通信协议则用于保障请求与响应在网络中的可靠传输。通过对这些环节进行统一封装，RPC 框架能够在不同语言和运行环境之间提供一致的调用体验。

在分布式系统中，RPC 技术还常与服务发现、负载均衡和容错机制相结合，以提升系统的可用性和扩展性。通过动态路由和调用调度，RPC 框架可以在多个服务实例之间分发请求，在部分节点发生故障时自动进行切换，从而保障服务调用的连续性。这种能力使 RPC 成为构建高可用分布式服务体系的重要基础。

总体而言，RPC 技术通过将网络通信抽象为过程调用，为分布式系统提供了一种高效、清晰的服务交互方式。其在接口化设计、通信封装以及调用治理等方面的工程实践，为现代服务化架构和复杂业务系统的构建提供了重要支撑。

2.2 文档数据库 MongoDB

MongoDB 是一种面向文档的 NoSQL 数据库，采用灵活的文档数据模型，广泛应用于内容管理、日志存储以及高并发读写等业务场景。MongoDB 以 BSON（Binary JSON）格式存储数据，能够自然映射应用层对象结构，支持对结构化与半结构化数据的统一存储，具有模式灵活、扩展性强等特点，在互联网和企业级系统中得到广泛应用。

在数据组织方式上，MongoDB 以数据库（Database）和集合（Collection）作为数据管理的基本单位，每条数据以文档形式存储在集合中。与传统关系型数据库相比，MongoDB 不强制要求固定的表结构，不同文档可以包含不同字段，从而能够灵活适应业务数据结构的演进需求。这种文档式存储方式在内容频繁变化或字段不固定的场景下具有较高的适用性。

在查询与索引机制方面，MongoDB 提供了丰富的查询接口，支持基于字段的精确查询、范围查询以及条件组合查询等操作。系统支持为文档字段建立多种类型的索引，包括单字段索引、复合索引和全文索引等，以提升查询性能。同时，MongoDB 提供聚合管道机制，能够对集合中的数据进行过滤、分组和统计分析，满足一定程度的数据分析需求。

在系统架构层面，MongoDB 支持副本集（Replica Set）和分片（Sharding）机制。副本集通过主从复制提高数据的可用性和容错能力，当主节点发生故障时能够自动完成主节点切换；分片机制则通过将数据分布到多个节点上，实现数据的水平扩展，支持大规模数据存储和高并发访问场景。

在分布式应用中，MongoDB 常用于承载对结构灵活性要求较高的数据存储需求，适合存储内容元数据、非结构化信息以及业务过程数据。其灵活的数据模型和良好的扩展能力，使其能够在业务规模增长和数据结构演进过程中保持较高的适应性，为复杂系统的数据管理提供稳定支撑。

2.3 消息队列 RocketMQ

Apache RocketMQ 是一种高性能、高可靠的分布式消息中间件，最初由阿里巴巴提出并开源，现已成为 Apache 顶级项目，广泛应用于异步通信、系统解耦和流量控制等业务场景。RocketMQ 采用发布—订阅（Publish-Subscribe）的消息模型，支持点对点和广播等多种消费方式，能够满足大规模分布式系统

中对消息传递可靠性和稳定性的要求。

在消息组织方式上，RocketMQ 以主题（Topic）作为消息管理的基本单位，消息在 Broker 中按照队列（Queue）进行存储和分发。通过将同一主题下的消息分布到多个队列中，系统可以实现消息的并行写入和并行消费，从而提升整体吞吐能力。在消息存储方面，RocketMQ 采用顺序写磁盘的方式进行消息持久化，在保证可靠性的同时提高了写入效率。

在消息投递与消费机制方面，RocketMQ 支持生产者向指定主题发送消息，由消费者以订阅方式进行消费，并通过消费进度管理和投递确认机制保障消息的可达性。系统支持至少一次（At-Least-Once）的消息投递语义，并提供顺序消息和延迟消息等能力，使其能够适应对消息时序性和定时触发有要求的业务场景。

在系统架构层面，RocketMQ 由 NameServer、Broker、Producer 和 Consumer 等组件构成。NameServer 负责维护集群的路由信息，Broker 负责消息的存储与转发，Producer 和 Consumer 分别承担消息的生产与消费职责。该架构采用去中心化设计，支持 Broker 节点的动态扩展和故障转移，在节点异常情况下能够保证消息服务的连续性。

在分布式系统中，RocketMQ 常用于构建基于事件驱动的消息通信机制，通过消息队列对请求进行缓冲和调度，降低系统之间的直接依赖关系。同时，消息队列能够在高并发场景下对突发流量进行削峰填谷，并通过消费速率控制实现限速处理，从而提升整体系统的稳定性和可扩展性。

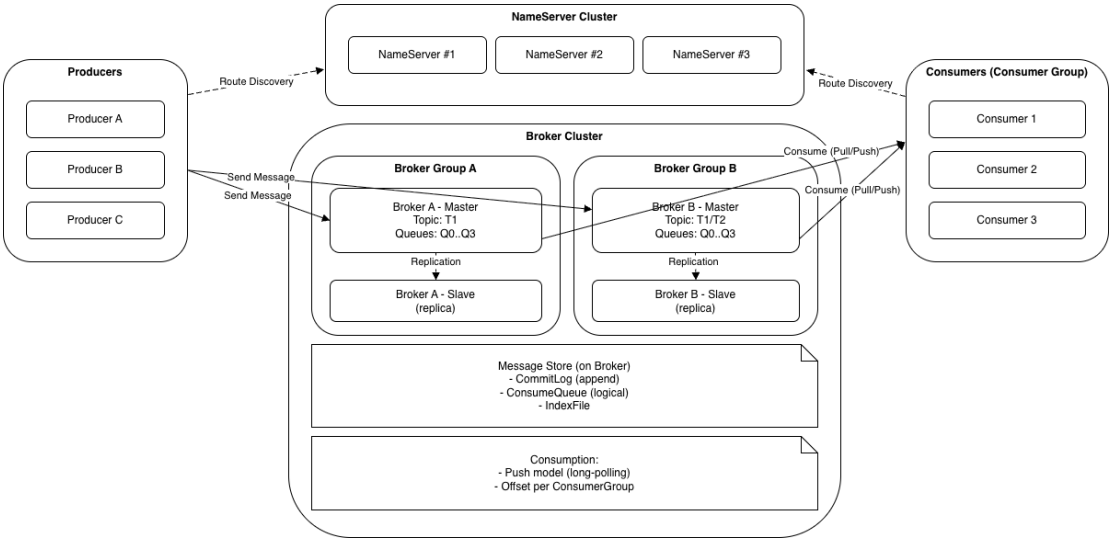

图 2-1: RocketMQ 架构图

2.4 全文搜索引擎 ElasticSearch

ElasticSearch 是一种基于 Lucene 构建的分布式搜索与分析引擎，广泛应用于全文检索、日志分析和实时数据查询等场景。该系统采用面向文档的数据模型，支持对结构化、半结构化以及非结构化数据的统一存储与检索，具有高并发、低延迟和良好可扩展性等特点，已成为内容管理与检索系统中的重要基础组件。

在数据组织方式上，ElasticSearch 以索引作为数据管理的基本单位，每个索引由多个分片组成，分片是实际存储和查询的最小单元。通过将分片分布到不同节点上，系统能够实现查询请求的并行处理，从而在数据规模较大时仍保持较高的查询性能。在文本检索方面，ElasticSearch 基于倒排索引结构对文档内容进行索引，将词项与文档之间建立映射关系，使系统能够高效支持关键词匹配、短语查询以及相关性计算等全文检索功能。

在查询机制方面，ElasticSearch 提供了基于 DSL 的查询方式，支持精确匹配、全文匹配、范围查询以及多条件组合查询等多种检索模式。在全文检索场景下，系统通常采用 BM25 等相关性算法对文档进行评分，并按照相关性得分对查询结果进行排序，以优先返回与查询条件匹配度较高的内容。同时，ElasticSearch 支持在查询过程中结合过滤条件和聚合分析，实现对内容数据的统计与分析，满足复杂业务场景下的检索与分析需求。

在系统架构层面，ElasticSearch 通过主分片与副本分片机制保障数据的高可用性，当部分节点发生故障时，系统能够自动完成分片迁移和主从切换，保证服务的连续性。此外，ElasticSearch 支持节点的动态扩缩容，能够根据业务数据规模和访问压力灵活调整集群规模，具备良好的横向扩展能力，适合内容规模持续增长的业务场景。

在内容管理与应用场景中，ElasticSearch 非常适合用于支撑基于关键词和结构化条件的内容检索需求。一方面，其全文检索能力能够支持用户快速定位相关内容；另一方面，通过对内容类型、标签、类目等字段建立索引并进行组合查询，可以实现内容筛选、聚合展示等功能。在本文所设计的平台中，ElasticSearch 主要用于承载内容的全文检索与结构化检索能力，与向量检索等技术形成互补，为内容的页面化展示和智能化应用提供高效、稳定的基础检索支撑。

2.5 Embedding 与向量检索技术

Embedding 技术是一种将文本、图片等非结构化数据映射为高维向量表示的方法，其核心目标是将语义信息编码到向量空间中，使语义相近的内容在向量空间中距离更近。通过 Embedding，原本难以直接进行数学计算的自然语言或多模态内容可以被转换为统一的数值表示形式，从而为后续的相似度计算和检索提供基础。在近年来的自然语言处理和信息检索领域中，Embedding 已成为语义理解和语义匹配任务中的关键技术。

在内容表示层面，Embedding 模型通常基于深度学习方法，对文本中的上下文语义进行建模，将词语、句子或文档映射为固定维度的向量。与传统基于关键词的表示方式相比，Embedding 能够在一定程度上捕捉语义相似性，即使不同文本在表面词汇上存在差异，其向量表示仍可能较为接近。这一特性使得 Embedding 在处理同义表达、语义扩展等场景中具有明显优势，适合用于复杂内容语义建模。

在 Embedding 的基础上，向量检索技术用于解决在大规模向量集合中高效查找相似向量的问题。向量检索的基本思想是，通过定义向量之间的相似度度量方式，对查询向量与候选向量进行比较，从而返回语义上最为接近的结果。在实际应用中，常用的相似度度量方式包括余弦相似度和向量内积等，这些方法能够较好地反映向量之间的语义接近程度。

随着内容规模的不断增长，简单的全量向量比对在性能上难以满足实际需求，因此向量检索系统通常通过构建专门的索引结构来提升查询效率。通过对向量空间进行组织和近似搜索，系统可以在保证一定召回效果的前提下显著降低检索开销，从而支持大规模数据场景下的实时相似度查询。这类索引化的向量检索机制已成为现代语义搜索系统中的重要组成部分。

在内容管理与知识应用场景中，Embedding 与向量检索技术能够为内容提供基于语义的检索能力，使系统不再局限于关键词匹配，而是能够从语义层面发现相关内容。通过将内容表示为向量并引入向量检索机制，系统可以在内容规模较大、表达方式多样的情况下，仍然实现较高质量的相关内容召回，为后续的智能应用奠定基础。

2.6 检索增强生成 RAG

检索增强生成（Retrieval-Augmented Generation，RAG）是一种将信息检索与生成模型相结合的应用范式，旨在提升生成模型在知识密集型任务中的准确性和可靠性。与仅依赖模型参数进行生成的方式不同，RAG 在生成阶段引入外部知识检索过程，使生成模型能够基于检索到的相关内容进行回答，从而在一定程度上缓解大语言模型在知识更新滞后和事实幻觉等方面的局限。

RAG 技术的提出源于大语言模型在实际应用中的局限性。尽管预训练模型在语言理解和生成能力上取得了显著进展，但其内部知识主要来源于训练阶段的静态语料，难以覆盖最新或特定领域的知识，同时在面对知识密集型问题时容易生成与事实不一致的内容。为解决上述问题，研究者开始探索将外部知识源引入生成过程的技术路径。在这一背景下，Patrick Lewis 等人在 2020 年提出了 RAG 框架，该工作首次系统性地将信息检索模块与生成模型进行结合，通过在生成过程中引入外部文档作为条件输入，显著提升了知识密集型任务的回答效果，为后续相关研究奠定了基础。

在 RAG 的基本流程中，系统通常首先根据用户输入执行检索操作，从外部知识库中召回与问题语义相关的内容；随后，将检索结果与用户输入一并作为上下文信息提供给生成模型，由模型综合多条检索信息生成最终回答。通过这种“先检索、后生成”的方式，RAG 将外部知识的可控性与生成模型的表达能力结合起来，使生成结果不再完全依赖模型参数中隐含的知识。

随着语义表示和向量检索技术的发展，RAG 架构逐步从基于关键词的检索方式演进为基于语义的检索方式。通过引入 Embedding 和向量检索机制，系统能够在更大规模和更复杂的知识库中高效获取语义相关内容，提高检索结果的相关性和覆盖度。这一阶段的研究重点逐渐从“是否引入检索”转向“如何提升检索质量与生成效果之间的协同能力”，推动 RAG 从概念性方法演进为具有工程可行性的系统范式。

在工程实践中，RAG 技术已被广泛应用于知识问答、智能客服和文档理解等场景。相较于传统基于检索的问答系统，RAG 不再直接返回检索结果，而是通过生成模型对多条信息进行综合理解和组织，生成更加连贯和自然的回答；相较于纯生成式模型，RAG 通过引入外部知识来源，提高了生成结果的可解释性和可控性，使系统能够更好地适应知识更新频繁和领域知识复杂的应用环境。

总体而言，RAG 作为一种将信息检索与生成模型相结合的系统级应用范式，为智能内容应用提供了有效的技术路径。通过在生成过程中引入外部知识检索机制，RAG 在准确性、灵活性和可扩展性之间取得了较好的平衡，已成为当前知识密集型应用和企业级智能问答系统中的重要技术方向。

2.7 本章小结

本章主要介绍了远程过程调用 RPC、文档数据库 MongoDB、消息队列 RocketMQ、全文搜索引擎 ElasticSearch、Embedding 与向量检索技术以及检索增强生成（RAG）技术的基本原理和应用场景。其中，RPC 技术是一种用于分布式系统中实现跨进程或跨节点服务通信的技术机制；MongoDB 是一种基于文档模型的 NoSQL 数据库，适用于存储结构灵活、模式多变的数据；RocketMQ 是一种高性能的分布式消息中间件，常用于异步通信、系统解耦和流量控制；ElasticSearch 是一种分布式全文搜索与分析引擎，能够支持大规模内容的高效检索与统计分析；Embedding 与向量检索技术通过语义表示和相似度计算，实现了基于语义的内容召回能力；RAG 技术则通过将信息检索与生成模型相结合，为知识密集型应用提供了更为准确和可控的生成式能力。这些技术在内容管理、内容检索以及智能应用等方面发挥着重要作用，为本文后续系统的需求分析、架构设计与功能实现提供了必要的技术支撑。

第三章　内容管理、应用与治理平台的分析与设计

3.1　系统整体概述

面向广告与商业化内容的管理、应用与治理平台是一个以内容资产全生命周期为核心的综合性平台系统，目标是在内容规模持续增长、业务线与应用场景不断扩展的背景下，实现内容的统一建模、规范化管理、高效应用以及质量可控。平台以“内容资产化”和“能力平台化”为设计导向，将内容从分散的业务沉淀与人工维护过程，转化为可组织、可复用、可追踪、可治理的系统化资产，并通过统一接口对上游业务系统、智能助手与运营工具提供稳定服务。

从业务视角看，平台服务于广告与商业化生态中的多类使用场景，包括业务人员的知识获取与运营决策支持、面向外部的产品说明与规范指引、以及客户服务和智能问答等高频咨询场景。上述场景对内容提出了共同要求：一是内容来源多、形态多样，且跨业务线复用需求强；二是内容更新频繁，依赖人工维护易产生不一致与过期；三是内容消费方式从“页面浏览”逐步演进到“搜索与问答驱动”，要求更高的检索质量与可解释性；四是内容质量问题需要可度量、可触发、可闭环，否则难以支撑规模化运营。因此，本平台在总体设计上采用“管理与应用一体、治理闭环驱动”的思路，以内容组织与权限体系为基础，以检索与智能问答能力为牵引，以质量治理和数据洞察为反馈闭环，实现内容体系的持续演进。

从系统视角看，平台整体由内容管理、内容应用、内容治理与数据洞察四个核心域组成，并通过统一的数据模型、权限体系与事件机制实现协同。内容管理域负责内容的生产与维护，包括空间与应用的分层组织、内容形态管理（文章、问答、课程、案例等）、权限控制与审批发布等能力。该域的核心作用是将多来源、多形态内容纳入统一规范，形成可控的内容资产底座，并为后续检索、问答与治理提供一致的数据输入和业务语义。

内容应用域面向内容消费与智能化使用需求，提供两类能力：其一是面向

页面化场景的内容展示与检索能力，通过统一内容接口支撑列表、目录导航、详情页、关键词搜索、标签聚合等常见页面组件；其二是面向智能问答场景的检索增强生成能力，通过内容切片、索引构建、多通道召回与重排等手段，为上游智能应用提供高相关性、可追溯的内容依据。与传统“内容库 + 搜索”不同，平台在应用域强调“可控检索”和“结果可解释”，即召回结果需可定位到具体内容片段与来源，以便在治理与迭代中形成明确反馈路径。

内容治理域面向内容规模化带来的质量问题，构建“自动检测—任务化分发—人工处理—结果回收”的闭环机制。治理能力覆盖重复度、歧义度与覆盖度等典型问题：重复度治理用于识别内容冗余与重复维护；歧义度治理用于检测同一问题下的内容冲突、不一致或表述歧义；覆盖度治理用于从知识体系与用户需求侧识别内容缺口与薄弱点。平台将治理结果以标准化的治理任务形式落地，包括问题定位、影响范围、建议处理方式与责任归属，配合任务状态流转与审计能力，使治理过程可追踪、可度量、可持续运营，从而避免治理仅停留在离线分析或一次性专项。

数据洞察域承担平台运行效果与治理成效的量化评估职责，通过对页面访问、用户会话、检索与问答链路日志、内容使用情况等数据进行统计分析，输出面向运营与产品迭代的指标体系。该域既用于评价内容应用效果（如内容被召回次数、生成引用次数、转人工率等），也用于反哺治理策略（如高频问题聚类、低命中内容识别、质量问题分布等），从而形成“应用驱动生产、数据反哺治理”的闭环运营机制。

在关键流程层面，平台围绕内容全生命周期形成若干核心链路：在内容生产链路中，内容经创建与编辑后进入审批与发布流程，发布版本作为对外可用的内容资产进入索引与应用链路；在索引构建链路中，平台对不同内容形态进行标准化处理（如切片、结构化字段抽取、元数据归一）并构建全文索引与向量索引，以支撑检索与问答；在内容消费链路中，用户通过页面浏览、搜索或智能问答触发内容召回，平台返回内容或为生成模型提供依据；在质量治理链路中，平台周期性或事件驱动地对内容进行质量检测，将发现的问题沉淀为治理任务并推动处理，处理结果再反向影响内容库与索引；在评估反馈链路中，数据洞察持续沉淀关键指标与行为数据，为内容生产策略、检索策略与治理规则提供优化依据。

总体而言，本平台以统一内容资产底座为核心，将内容管理、内容应用、内容治理与数据洞察四个域有机整合：内容管理域负责内容的生产与规范化，

为后续应用与治理提供一致的数据输入；内容应用域面向不同消费场景提供展示、检索与问答能力，实现内容价值的直接输出；内容治理域通过质量检测与任务化闭环，保障内容体系的持续演进；数据洞察域则通过效果评估与问题定位，形成”应用驱动生产、数据反哺治理”的反馈机制。四个域相互协同、闭环迭代，既覆盖传统内容平台的基础管理需求，也面向智能化内容消费方式提供支撑，为广告与商业化业务在知识沉淀、信息一致性以及智能化服务能力方面提供稳定、可扩展的工程化方案。

3.2 内容管理、应用与治理平台需求分析

3.2.1 用户角色

本平台的用户角色如表3-1所示，主要分为四种，分别是：平台负责人、业务负责人、业务运营和客户。平台负责人负责整个平台层面的配置与运维管理；业务负责人负责某一业务空间下内容体系的整体管理与维护；业务运营是平台内容生产和日常运营的主要执行角色；客户主要作为内容的消费方，通过页面或智能服务获取平台提供的内容支持。

表 3-1: 用户角色表

用户角色	描述
平台负责人	平台的最高管理角色，负责整个平台的基础配置、空间管理及全局权限控制，保障平台稳定运行。
业务负责人	负责某一业务线或业务空间下内容体系的管理与运维，对内容结构、应用配置及业务权限进行整体把控。
业务运营	负责平台内容的创建、编辑和维护等日常运营工作，并在权限范围内查看相关内容使用数据。
客户	平台内容的消费角色，包括广告主、代理商等，通过页面浏览、搜索或智能助手等方式使用平台内容。

除上述用户角色外，平台还需要与上游智能应用（如 AI 助手）进行接口对接。该类调用方通常以系统身份访问平台的内容检索与问答能力，其行为数据将被纳入会话分析与效果评估范围。

3.2.2 内容管理功能需求

内容管理功能的用例图如图3-1所示。内容管理模块作为平台的核心模块，主要负责内容资产的全生命周期管理，包括内容的生产、维护、组织、权限控制以及发布上线等环节。该模块旨在将分散在不同业务线和系统中的内容进行统一纳管，通过标准化的模型和流程，确保内容在流转过程中的一致性、安全性和可控性。

内容管理用例图占位（待补充）

图 3-1: 内容管理用例图

内容管理模块的功能需求可进一步拆分为多个相互衔接的功能块，以覆盖内容从组织边界建立、内容生产维护到权限约束与发布上线的全链路。各功能块共同构成内容资产的可管、可控与可追溯能力。为保证论述结构清晰，本节按照功能块依次给出需求说明，并为每一类子需求列举 1–3 个代表性用例。

（1）空间与应用管理。平台需要支持以空间为单位建立业务隔离与治理边界。空间用于承载内容资产的业务归属、成员范围与默认权限策略，并支持空间的创建、编辑以及基础配置维护。为适配不同消费场景，平台需在空间内支持应用配置：应用定义内容对外呈现的组织方式与可用能力，包括可用内容形态、类目树与标签体系、检索范围与排序规则等。空间与应用管理功能主要包含创建空间与创建应用等用例。其中，创建空间与创建应用用例描述分别如表3-2与表3-3所示。

表 3-2: 创建空间用例描述表

ID	CM-05
用例名称	创建空间
参与者	平台负责人
用例描述	创建业务空间以作为内容资产的组织边界与权限边界，并初始化空间基础配置。
触发条件	新业务线接入或现有业务需要建立独立治理边界。
前置条件	1. 当前用户具备空间管理权限。
后置条件	1. 系统生成空间 ID 并创建默认成员组与默认权限策略。2. 记录空间创建审计日志。
基本流程	1. 用户进入空间管理页面并点击新建空间。2. 系统弹出空间信息编辑弹窗。3. 用户填写空间名称、业务归属等信息并保存。4. 系统校验信息并持久化保存空间配置。
替代流程	1. 若空间名称重复或信息校验失败，系统提示原因并要求用户修改。

表 3-3: 创建应用用例描述表

ID	CM-06
用例名称	创建应用
参与者	业务负责人
用例描述	在指定空间内创建应用并配置内容组织方式，以适配不同内容消费场景。
触发条件	业务需要新增内容门户、帮助中心或面向智能应用的知识库入口。
前置条件	1. 目标空间已存在且当前用户具备应用管理权限。
后置条件	1. 系统生成应用 ID 并初始化应用配置（可用形态、类目树与标签配置等）。2. 记录应用创建审计日志。
基本流程	1. 用户进入空间详情页并点击创建应用。2. 用户选择应用类型并填写应用名称与基础配置。3. 系统校验配置并创建应用。4. 系统返回应用创建成功提示并展示应用配置入口。
替代流程	1. 若应用配置不完整（如未配置可用内容形态），系统提示并阻止创建。

（2）多形态内容服务。平台需要支持多种内容形态的统一纳管与一致化维护。不同内容形态在结构与消费方式上存在差异，但应具备统一的基础管理能力，包括创建、编辑、保存草稿、版本管理与变更记录等。多形态内容服务的典型用例包括创建内容、编辑内容与删除内容，其用例描述分别如表3-4、表3-5与表3-6所示。

表 3-4: 创建内容用例描述表

ID	CM-01
用例名称	创建内容
参与者	业务运营
用例描述	在指定空间与应用下创建新的内容实体，支持 knowledge、faq、course、case 等多种形态，并生成初始草稿版本。
触发条件	业务需要新增产品说明、规则解读或运营素材等内容资产。
前置条件	1. 目标空间与应用已创建并处于可用状态。2. 当前用户拥有该空间下的内容创建权限。
后置条件	1. 系统生成唯一的内容 ID。2. 内容状态置为“草稿”。3. 记录创建操作日志。
基本流程	1. 用户进入内容管理列表页，点击“新建”按钮。2. 用户选择内容形态（如文章、问答等）。3. 系统初始化空白草稿并跳转至编辑器。4. 用户填写标题、正文及必要的属性字段（如类目、标签）。5. 用户点击“保存”按钮。6. 系统校验必填项与格式，校验通过后持久化存储。
替代流程	1. 若校验失败，系统提示具体错误字段，用户修改后可重新保存。

表 3-5: 编辑内容用例描述表

ID	CM-02
用例名称	编辑内容
参与者	业务运营
用例描述	对已存在的内容进行修改，包括正文更新、属性调整或版本回滚，支持多人协同编辑与锁机制。
触发条件	内容信息发生变更或需要优化现有内容。
前置条件	1. 内容处于可编辑状态（如草稿、驳回或已发布）。2. 当前用户拥有该内容的编辑权限。
后置条件	1. 内容更新为新的草稿版本。2. 记录编辑操作日志与版本变更记录。
基本流程	1. 用户在内容列表点击目标内容的“编辑”按钮。2. 系统加载内容最新版本数据并进入编辑器。3. 用户修改正文或属性信息。4. 用户点击“保存”或“提交”。5. 系统保存修改并更新版本号。
替代流程	1. 若内容正在被其他用户编辑，系统提示“锁定中”并禁止并发修改。

表 3-6: 删除内容用例描述表

ID	CM-03
用例名称	删除内容
参与者	业务运营、业务负责人
用例描述	将不再使用的内容及其关联版本进行逻辑删除或物理删除，并清理相关索引与关联关系。
触发条件	内容过期、废弃或创建错误。
前置条件	1. 内容未处于“审批中”状态。2. 当前用户拥有删除权限。
后置条件	1. 内容状态置为“已删除”或从数据库移除。2. 触发相关索引的删除事件。
基本流程	1. 用户选中一条或多条内容，点击“删除”按钮。2. 系统弹出二次确认弹窗，提示删除后果。3. 用户确认删除。4. 系统执行删除操作，并同步清理关联资源。
替代流程	1. 若内容已被其他应用引用，系统提示“存在引用关系”，阻止删除或提示解除引用。

（3）权限管理。平台需要构建基于角色的访问控制（RBAC）体系，在空间与应用维度实现权限隔离与精细化授权。权限粒度应覆盖模块级与操作级，既能控制用户进入某一空间/应用的访问权限，也能控制内容的创建、编辑、删除、提审、审批、上线与下线等具体操作权限。权限管理功能主要包含成员角色分配与权限策略配置等用例，其用例描述如表3-7与表3-8所示。

表 3-7: 成员角色分配用例描述表

ID	CM-07
用例名称	成员角色分配
参与者	平台负责人，业务负责人
用例描述	为目标用户在空间或应用维度分配角色，以确定其可访问范围与可执行操作集合。
触发条件	新成员加入空间/应用或成员职责变更需要调整权限。
前置条件	1. 目标空间与应用已存在。2. 当前用户具备成员管理权限。
后置条件	1. 系统更新用户角色映射并立即生效。2. 记录授权变更审计日志。
基本流程	1. 管理员进入空间或应用的成员管理页面。2. 管理员选择目标用户并设置角色。3. 系统校验管理权限与角色合法性。4. 系统保存变更并返回成功提示。
替代流程	1. 若当前用户无权限执行授权操作，系统拒绝并提示原因。

表 3-8: 权限策略配置用例描述表

ID	CM-08
用例名称	权限策略配置
参与者	平台负责人
用例描述	配置空间或应用的默认权限策略与操作权限集合，实现对模块与操作级权限的精细化控制。
触发条件	业务治理要求调整默认权限口径或新增权限项。
前置条件	1. 当前用户具备全局权限配置能力。
后置条件	1. 系统更新权限策略并对后续鉴权生效。2. 记录权限策略变更审计日志。
基本流程	1. 用户进入权限配置页面选择目标空间/应用。2. 用户配置角色与权限项映射关系并保存。3. 系统校验配置合法性并持久化保存。
替代流程	1. 若配置导致关键角色缺失必要权限，系统提示风险并阻止保存。

（4）审核与发布。内容对外可用的前提是准确性与合规性可控。平台需要提供流程化的审核与发布机制，对内容状态流转进行约束。内容在生命周期中至少应包含草稿、待审批、已发布、已下线等状态，状态变更需满足权限校验与流程条件。审核与发布功能主要包含提交审核、内容审批与内容上下线等用例，其用例描述分别如表 3-9、表 3-10与表 3-11所示。

表 3–9: 提交审核用例描述表

ID	CM-09
用例名称	提交审核
参与者	业务运营
用例描述	将内容草稿提交进入审批流程，申请生成可对外使用的发布版本。
触发条件	草稿内容完成编辑，需要上线以支撑页面展示、检索或智能问答。
前置条件	1. 内容处于可提交状态且基础校验通过。2. 当前用户具备提审权限。
后置条件	1. 系统创建审批记录并将内容状态更新为待审批。2. 记录提审操作历史并通知审批者。
基本流程	1. 用户在内容详情页点击提交审核。2. 系统执行内容完整性校验与权限校验。3. 系统创建审批单并将内容状态置为待审批。4. 系统向用户返回提审成功提示。
替代流程	1. 若权限校验失败，系统拒绝提交并提示原因。2. 若内容校验失败，系统提示需要补全或修正的字段。

表 3-10: 内容审批用例描述表

ID	CM-10
用例名称	内容审批
参与者	业务负责人，审核员
用例描述	对待审批内容进行审核并给出通过或拒绝结论，决定内容能否生成发布版本并对外可见。
触发条件	审批者在审批列表中发现待处理任务或收到待审批通知。
前置条件	1. 内容状态为待审批。2. 当前用户具备审批权限。
后置条件	1. 若审批通过，系统生成发布版本并将内容状态更新为已发布，同时发布内容变更事件供下游消费。2. 若审批拒绝，系统将内容状态更新为未通过并记录拒绝原因。
基本流程	1. 审批者进入审批列表并打开目标审批单。2. 审批者查看内容关键信息、变更点与历史版本。3. 审批者选择通过或拒绝并提交意见。4. 系统更新审批单状态并回写内容状态。
替代流程	1. 若审批者选择拒绝，系统要求填写拒绝原因并通知发布者。

表 3-11: 内容上下线用例描述表

ID	CM-04
用例名称	内容上下线
参与者	业务负责人、业务运营
用例描述	控制内容在应用端的可见性。上线即发布内容使其对外可见，下线即撤回发布内容。
触发条件	内容审核通过需发布，或内容失效需紧急下线。
前置条件	1. 上线前需通过审批流程。2. 下线需具备相应管理权限。
后置条件	1. 更新内容发布状态（已发布/已下线）。2. 触发索引更新事件，同步搜索与问答端数据。
基本流程	1. 用户对“待发布”内容点击“上线”，或对“已发布”内容点击“下线”。2. 系统校验当前状态与权限。3. 系统更新内容状态。4. 系统发送状态变更消息至下游模块（搜索、网关等）。
替代流程	1. 若下线内容存在高频访问，系统记录下线原因备查。

（5）其他功能。除核心链路外，平台还需提供辅助能力以提升内容组织效率与复用效果，包括类目管理、标签管理以及应用引用机制等。其他功能主要包含类目管理、标签管理与应用引用内容等用例，其用例描述分别如表 3-12、表 3-13与表 3-14所示。

表 3-12: 类目管理用例描述表

ID	CM-11
用例名称	类目管理
参与者	业务负责人，业务运营
用例描述	在应用维度维护类目树结构，用于构建内容导航与主题聚合入口。
触发条件	新增业务主题或需要调整内容导航结构。
前置条件	1. 目标应用已存在且当前用户具备类目管理权限。
后置条件	1. 系统更新类目树配置并记录操作历史。
基本流程	1. 用户进入应用配置中的类目管理页面。2. 用户新增、编辑或调整类目节点层级。3. 系统校验类目结构合法性并保存。
替代流程	1. 若调整导致类目形成循环或层级不合法，系统提示并阻止保存。

表 3-13: 标签管理用例描述表

ID	CM-12
用例名称	标签管理
参与者	业务运营
用例描述	维护标签体系并将标签应用于内容，以支持跨类目的聚合检索与运营配置。
触发条件	需要新增运营标签、调整标签口径或清理废弃标签。
前置条件	1. 目标应用已存在且当前用户具备标签管理权限。
后置条件	1. 系统更新标签集合并记录变更历史。
基本流程	1. 用户进入标签管理页面。2. 用户新增、编辑或禁用标签。3. 系统校验标签名称与冲突并保存。
替代流程	1. 若标签已被大量内容使用且需要禁用，系统提示影响范围并要求二次确认。

表 3-14: 应用引用内容用例描述表

ID	CM-13
用例名称	应用引用内容
参与者	业务负责人，业务运营
用例描述	在应用维度引用其他应用或其他空间下的已发布内容，使其在本应用中可见与可用，并记录引用关系以支撑复用与影响范围分析。
触发条件	需要将权威口径内容在多个应用复用，避免重复维护与口径不一致。
前置条件	1. 被引用内容已发布且未下线。2. 当前用户具备应用配置权限并对被引用内容具备查看权限。
后置条件	1. 系统记录应用-内容引用关系并使引用在应用侧生效。2. 记录引用操作日志以便审计与追溯。
基本流程	1. 用户进入应用配置页面选择“内容引用”。2. 用户选择引用来源（空间/应用）并检索目标内容。3. 用户确认引用并保存配置。4. 系统校验权限与内容状态后创建引用关系并返回成功提示。
替代流程	1. 若内容未发布或权限不足，系统拒绝引用并提示原因。

3.2.3 内容应用功能需求

内容应用功能的用例图如图3-2所示。内容应用模块的主要功能是对内容进行展示、检索与问答，以满足不同场景下的内容消费需求。一方面，平台需要支持面向门户与管理后台的页面化展示与检索能力，帮助用户通过目录导航、列表筛选与关键词搜索快速定位内容；另一方面，平台需要面向智能问答等上游智能应用提供检索增强生成能力，使用户能够以对话方式获取结构化且可追溯的内容依据。

内容应用用例图占位（待补充）

图 3-2: 内容应用用例图

内容应用模块的核心用例可归纳为以下几类。

（1）内容展示。面向页面化消费场景，平台需提供内容目录与列表能力，支持按空间、应用、类目与标签等维度进行内容聚合展示，并提供详情页渲染与内容关联跳转能力，保障内容在不同业务线与不同应用中可被稳定复用。

（2）内容搜索。平台需提供关键词搜索与结构化筛选能力，支持对标题、正文、标签与类目等字段的组合查询，并保证检索结果在权限范围内可见。为提升体验，搜索结果需支持高亮、排序与分页等能力。

（3）智能问答。平台需提供面向智能问答场景的内容支撑能力：当用户以自然语言提问时，系统能够在指定空间与应用范围内检索相关内容并生成回答，同时返回可追溯的引用来源，以保障结果可解释并降低口径不一致风险。

本节选取三个关键用例进行描述。

表 3-15: 内容展示用例描述表

ID	CA-01
用例名称	内容展示
参与者	客户，业务运营
用例描述	用户在内容门户或管理后台按目录导航与列表聚合浏览内容，并查看内容详情。
触发条件	用户希望通过类目、标签或列表入口快速定位并阅读内容。
前置条件	1. 目标应用存在已发布内容。2. 用户具备对应空间与应用的访问权限。
后置条件	系统展示内容列表与详情，并记录访问行为数据用于后续分析。
基本流程	1. 用户进入内容门户或管理后台。2. 用户通过类目树或标签进入内容列表。3. 用户点击某一条内容进入详情页查看正文与关联信息。
替代流程	1. 若内容无权限访问或已下线，系统提示并返回可访问的替代内容或上级列表。

表 3-16: 内容检索用例描述表

ID	CA-02
用例名称	内容检索
参与者	客户，业务运营
用例描述	用户通过关键词与筛选条件检索内容，系统返回权限范围内的相关结果列表。
触发条件	用户需要定位某一产品说明、业务规则或典型案例。
前置条件	1. 目标空间与应用存在已发布内容。2. 用户具备对应空间与应用的访问权限。
后置条件	系统返回结果列表并记录检索行为数据用于后续分析。
基本流程	1. 用户进入内容门户或管理后台的搜索页面。2. 用户输入关键词并选择筛选条件。3. 系统执行权限过滤与检索召回。4. 系统按相关性与业务规则排序并返回结果。
替代流程	1. 若无匹配结果，系统提示并引导用户调整关键词或筛选条件。

表 3-17: 生成式问答用例描述表

ID	CA-03
用例名称	生成式问答
参与者	客户
用例描述	用户以自然语言提问，系统结合召回证据生成回答并返回引用来源。
触发条件	用户希望通过对话快速获取业务规则解释或操作指导。
前置条件	1. 用户具备问答入口的访问权限。2. 系统可获取足够的召回证据或具备降级策略。
后置条件	系统返回回答与引用，并记录会话明细与反馈信息用于评估。
基本流程	1. 用户在 AI 助手中输入问题。2. 系统完成内容召回与证据组织。3. 生成模型基于证据生成回答。4. 系统返回回答与引用来源。
替代流程	1. 若问题不在知识覆盖范围内，系统提示并引导用户转人工或查看相关内容列表。

3.2.4 内容治理功能需求

内容治理功能的用例图如图3-3所示。内容管理模块为平台提供了统一的内容资产底座，内容应用模块实现了内容的展示与智能问答，然而随着内容规模扩大与消费频率提升，重复冗余、表述冲突与覆盖不足等质量问题会直接影响检索效果与问答准确性。因此，内容治理模块的主要功能是对内容质量进行指标化检测，并以任务形式推动治理闭环，与前述模块协同形成”生产—应用—治理—反馈”的完整链路，保证内容体系在持续演进过程中的可维护性与可控性。

内容治理用例图占位（待补充）

图 3-3: 内容治理用例图

内容治理模块的核心用例包括治理检测、治理任务创建与分发、治理处理与审核、治理结果回收与追踪等。平台将治理问题抽象为标准化任务，任务中应包含问题类型、问题定位（内容与片段）、影响范围、建议处理方式与责任归属，从而使治理过程可追踪、可审计、可统计。

（1）重复度检测。识别内容体系中的重复条目与冗余片段，支持对重复内容进行聚合展示与合并建议，减少重复维护带来的成本与口径不一致风险。

（2）歧义度检测。针对同一问题或同一概念在不同内容中的表述冲突与语义歧义进行检测，给出冲突片段与候选一致口径，降低智能问答场景下的误导概率。

（3）覆盖度检测。结合用户查询与会话数据识别内容缺口，定位高频问题无有效内容支撑的领域，并形成补充内容的治理建议。

本节选取三个关键用例进行描述。

表 3-18: 质量检测用例描述表

ID	CG-01
用例名称	质量检测
参与者	平台负责人
用例描述	系统对指定范围内内容执行重复度、歧义度或覆盖度检测，输出可定位的问题集合与治理建议。
触发条件	需要对内容体系的质量问题进行阶段性治理或例行巡检。
前置条件	1. 指定空间与应用范围可用。2. 已具备相似度计算、冲突判定或缺口识别能力。
后置条件	系统生成检测结果并可转化为治理任务。
基本流程	1. 用户在治理模块选择检测范围与检测类型。2. 系统执行对应检测并生成问题定位与影响范围。3. 系统输出问题集合与建议处理方式。
替代流程	1. 若检测范围过大导致资源不足，系统支持分批检测并输出进度。

表 3-19: 治理任务创建用例描述表

ID	CG-02
用例名称	治理任务创建
参与者	平台负责人
用例描述	将检测结果转化为可分发的治理任务，明确责任人与处理要求。
触发条件	检测结果需要进入治理闭环并推动相关人员处理。
前置条件	已产生可定位的质量问题集合。
后置条件	1. 系统创建治理任务并进入待处理状态。2. 系统通知责任人并记录任务审计信息。
基本流程	1. 用户在检测结果中选择问题条目并点击创建任务。2. 用户设置任务类型、优先级与责任人。3. 系统创建任务并生成问题定位与建议处理内容。
替代流程	1. 若同一问题已存在未完成任务，系统提示合并或追加处理。

表 3-20: 治理任务处理用例描述表

ID	CG-03
用例名称	治理任务处理
参与者	业务运营
用例描述	责任人根据任务定位对内容进行合并、修订、补充或下线，并提交处理结果。
触发条件	责任人收到治理任务并需要完成内容修订。
前置条件	1. 责任人具备对应内容的编辑权限。2. 任务处于可处理状态。
后置条件	1. 内容变更进入相应发布流程。2. 任务状态更新并记录处理结果。
基本流程	1. 责任人打开治理任务查看问题定位与建议。2. 责任人跳转至对应内容进行修订或合并。3. 责任人提交任务处理说明并完成。4. 系统更新任务状态并触发后续校验或复检。
替代流程	1. 若处理涉及多内容协同，系统支持拆分子任务并分别跟踪。

3.2.5 数据洞察功能需求

数据洞察功能的用例图如图3-4所示。内容管理模块负责内容的生产与发布，内容应用模块实现内容的消费与问答，内容治理模块推动质量问题的检测与闭环处理，而数据洞察模块则作为平台的”反馈中枢”，对上述模块的运行效果进行量化评估，支撑运营决策与平台迭代。该模块通过对页面访问、检索与问答链路、用户会话等数据进行采集与分析，输出面向运营与治理的指标体系，使平台能够形成”应用驱动生产、数据反哺治理”的闭环。

数据洞察用例图占位（待补充）

图 3-4: 数据洞察用例图

数据洞察模块的关键用例包括会话分析、内容效果评估与治理成效统计等。平台通过对页面访问、检索与问答链路日志进行采集与聚合，支撑运营人员从会话、内容与治理三个视角对平台效果进行量化评估，并为内容生产与治理策略调整提供依据。

表 3-21: 查看会话分析用例描述表

ID	DI-01
用例名称	查看会话分析
参与者	平台负责人，业务负责人
用例描述	用户查看检索与问答链路的会话统计与明细，分析内容命中与效果指标。
触发条件	需要评估某一空间/应用的内容效果或定位智能问答异常。
前置条件	已完成数据采集与指标聚合。
后置条件	无
基本流程	1. 用户进入数据洞察模块并选择时间范围与业务范围。2. 系统展示会话指标概览。3. 用户下钻查看会话明细与证据引用。
替代流程	1. 用户可按问题类型、命中率或转人工率筛选会话集合。

表 3-22: 查看内容效果用例描述表

ID	DI-02
用例名称	查看内容效果
参与者	业务负责人，业务运营
用例描述	用户从内容维度查看曝光、检索命中、问答引用等指标，评估内容可发现性与支撑价值。
触发条件	需要评估某一内容在不同消费场景中的使用效果并定位低效内容。
前置条件	已完成内容维度的日志采集与指标聚合。
后置条件	无
基本流程	1. 用户进入数据洞察模块并选择内容范围与时间范围。2. 系统展示内容效果指标列表。3. 用户点击某一内容下钻查看来源渠道与明细。
替代流程	1. 用户可按空间、应用、类目与标签等维度过滤内容集合。

表 3-23: 查看治理成效用例描述表

ID	DI-03
用例名称	查看治理成效
参与者	平台负责人，业务负责人
用例描述	用户查看治理任务完成情况与质量指标变化，评估治理策略有效性并指导下一轮治理。
触发条件	需要复盘阶段性治理专项或评估例行治理的收益。
前置条件	1. 治理任务已线上化流转并沉淀状态数据。2. 质量检测结果已指标化存储。
后置条件	无
基本流程	1. 用户进入数据洞察模块选择治理分析视图。2. 系统展示任务完成率、处理周期与问题分布。3. 用户下钻查看具体问题类型、责任归属与复检结果。
替代流程	1. 用户可按空间、应用或治理类型筛选查看。

3.2.6 系统非功能需求

在广告与商业化场景下，内容平台面临内容规模大、更新频繁、应用链路长且跨系统依赖多等特点，除功能需求外还需满足一系列非功能需求。

（1）性能与可用性。平台需满足高并发访问与高频内容更新场景下的响应要求，核心读链路应具备稳定低延迟特征；平台需具备高可用部署能力，关键服务支持故障隔离与快速恢复，避免单点故障影响内容消费与问答链路。

（2）一致性与可追溯性。内容在发布前后存在草稿、待审批、已发布等状态，平台需保证状态流转的正确性；对于检索与问答结果，需要能够追溯到具体内容来源与片段位置，以便问题定位与治理闭环。

（3）安全与权限。平台需对不同空间与应用进行权限隔离，严格控制内容的可见范围与操作权限，确保内容管理与对外开放接口符合安全要求；对关键操作需记录审计日志以满足合规与追责需求。

（4）可扩展性与可维护性。平台需支持内容形态、治理规则与数据指标的持续扩展，并尽量以配置化方式降低新增成本；模块边界清晰，便于独立迭代与部署。

（5）可观测性。平台需对内容发布、索引构建、召回与重排、治理任务流转等关键链路进行埋点与日志采集，形成统一的链路追踪与指标体系，支撑线上问题定位与效果评估。

3.3 系统整体设计

平台整体架构如图3-5所示。平台采用分层架构设计，从上到下依次为接入层、平台服务层、支撑服务层以及数据与基础设施层。分层设计的目标是将面向业务的能力与通用支撑能力解耦，使内容管理、应用、治理与洞察可以围绕统一内容资产底座协同演进，同时通过事件机制将发布后的索引、缓存与统计等处理解耦为异步链路，从而兼顾系统扩展性与稳定性。

在接入层，平台面向管理后台、内容门户以及上游 AI 助手与开放接口提供统一访问入口。通过 API 网关完成鉴权、路由与限流等通用能力，保障不同调用方在统一安全边界内使用平台服务。平台服务层由内容管理、内容应用、内容治理与数据洞察四个核心模块构成，是面向业务场景的能力输出中心。支撑服务层提供权限、审批、类目、标签、附件、索引构建与数据埋点等通用能力，为平台服务层提供可复用的基础支撑。数据与基础设施层承载结构化与非

结构化数据存储、检索与向量索引、消息队列以及动态配置等基础组件，为上层能力提供稳定的运行底座。

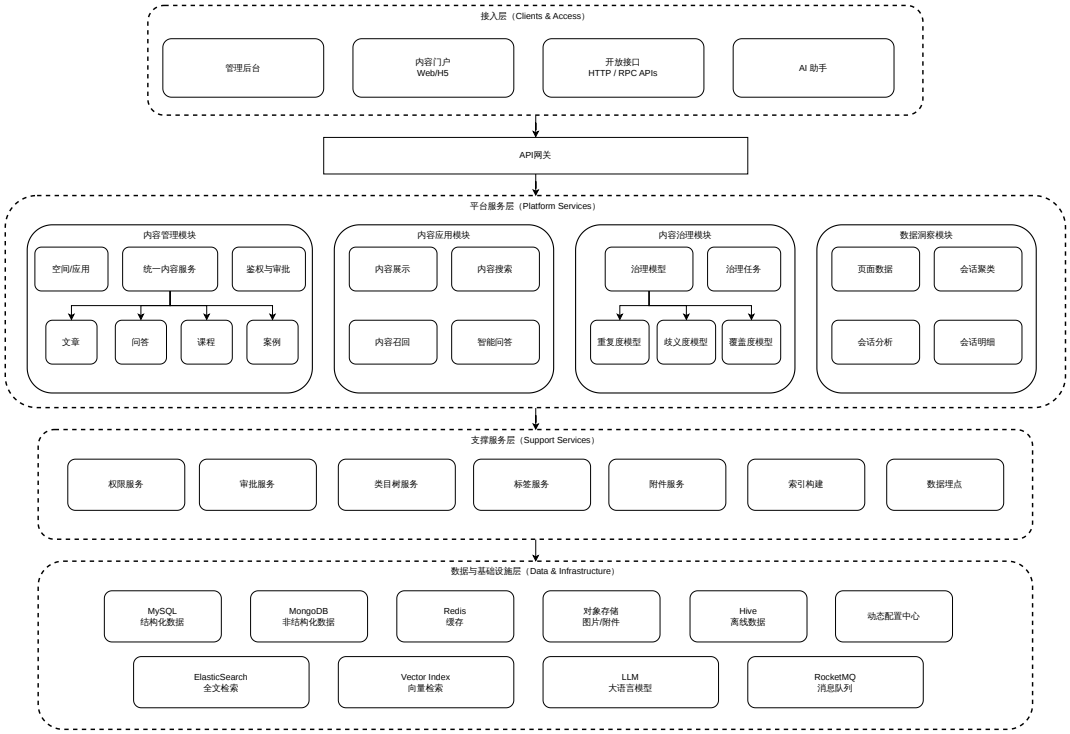

图 3–5: 系统架构图

为便于从不同角度理解平台的整体设计，本文在系统架构图基础上，引入逻辑视图、开发视图与物理视图对平台进行补充说明。逻辑视图强调业务能力与闭环关系，开发视图强调工程模块划分与依赖关系，物理视图强调部署形态与基础设施选型。

（1）逻辑视图。图 3–6为平台的逻辑视图，用于从业务能力与数据能力角度描述系统的主要功能模块及其协作关系。平台在逻辑上由内容管理、内容应用、内容治理与数据洞察四个核心域构成：内容管理负责空间与应用组织、内容生产维护、内容权限（content_auth）与审批发布，将 knowledge、faq、course 与 case 等多形态内容纳入统一内容资产；内容应用面向门户与管理后台提供展示与检索能力，并面向上游 AI 助手提供生成式问答能力，支持返回可追溯的引用来源；内容治理对内容质量进行重复、歧义与覆盖等维度的检测，并通过治理任务推动修订与复检；数据洞察对访问、检索与问答行为进行统计分析，评估内容效果与治理收益并反哺运营策略。四个核心域以统一内容模型作为数据输入口径，并通过发布事件与任务事件实现跨域协同，形成“生产—发

布—应用—治理—评估”的闭环体系。

系统逻辑视图占位（待补充）

图 3-6: 系统逻辑视图

（2）开发视图。图3-7为平台的开发视图，用于描述系统在工程实现层面的模块划分与依赖关系。平台对外提供管理后台、内容门户与开放接口等入口，统一由网关完成鉴权、路由与限流；核心业务能力按域拆分为内容管理、内容应用、内容治理与数据洞察等服务模块，并复用支撑服务层提供的通用能力，包括内容权限（content_auth）与角色授权、审批流转、类目与标签、附件管理、索引构建、埋点与任务调度等。为降低耦合与提升可维护性，平台将发布后的索引更新与统计采集等耗时操作以事件方式异步化处理：内容管理在发布通过后产生内容变更事件，下游索引构建服务消费事件并完成 document/slice 切片更新与索引刷新；数据洞察消费埋点数据生成会话与指标数据；内容治理在检测与任务处理后将修订结果回写内容管理并触发再次发布。该开发视图突出“核心域聚焦业务语义、支撑域复用通用能力、事件机制连接跨域链路”的设计思想。

开发视图占位（待补充）

图 3-7: 开发视图

（3）物理视图。图3-8为平台的物理视图，用于描述系统在部署与运行环境中的组件构成。平台的管理后台与内容门户以 Web 应用形式部署，并通过网关访问后端服务；后端核心服务以可独立扩容的服务实例部署，支撑高并发读链路与高频内容更新场景。数据存储方面，平台使用关系型数据库存储空间、应用、内容元数据、版本、内容权限（content_auth）、审批与治理任务等结构

化数据；案例（case）详情存储在 MongoDB 中以适配灵活字段；正文与附件等大对象以文档存储或对象存储方式管理。检索侧由全文检索与向量索引共同支撑，索引对象以 document 与 slice 为基本单位并在发布后通过增量任务更新。异步处理通过消息队列承载内容发布事件、索引任务事件与治理任务事件，保障主链路稳定并提高整体吞吐。数据洞察方面，平台在在线链路采集会话与行为数据，并依赖离线数仓进行聚合计算以产出报表指标。该物理视图体现了平台“多存储协同、事件驱动解耦、按需扩展”的工程部署思路。此外，为保障线上稳定性与问题可定位性，平台在运行过程中对发布、索引构建、问答与治理等关键链路进行日志与埋点采集，并将运行状态接入监控告警体系；同时通过动态配置能力对检索范围、阈值与开关策略进行配置化管理，降低策略调整对研发发布的依赖。

物理视图占位（待补充）

图 3–8: 物理视图

3.4 系统的模块设计

3.4.1 内容管理模块设计

内容管理模块是平台的内容资产底座，对应需求分析中的” 内容管理功能需求” 部分，负责空间与应用组织、内容生产与版本管理、内容权限（content_auth）控制以及审批发布等能力。该模块的设计目标是以统一数据模型纳管多形态内容，并通过可追溯的权限与流程机制，保障内容从生产到发布的过程可控、结果可查，为后续内容应用与内容治理提供一致的数据输入。以下对内容管理模块的主要功能子模块进行详细设计。

（1）空间与应用管理。空间与应用是内容管理模块的组织基础。平台采用“空间—应用”二级组织模型：空间作为业务归属与权限边界，应用作为内容消费场景与配置边界。空间管理主要包含空间创建、成员管理与默认策略配

置；应用管理主要包含应用创建、能力开关与内容范围配置等。空间与应用管理的关键交互过程如图3-9所示。

空间与应用管理时序图占位（待补充）

图 3-9: 空间与应用管理时序图

（2）多形态内容与版本管理。平台在内容层引入统一内容模型，使用 content_type 区分 knowledge、faq、course 与 case 四类内容形态，并通过版本表维护内容的版本链与变更记录。knowledge 更偏长文档组织与富文本编辑；faq 强调标准问与答案口径；course 由多个 lesson 组成并以课节顺序组织学习路径；case 的详情结构变化频繁，采用 MongoDB 以 JSON 详情字段承载大部分可变字段。内容创建、编辑与版本生成的交互过程如图3-10所示；内容状态流转的状态机示意如图3-11所示。

内容创建与版本管理时序图占位（待补充）

图 3-10: 内容创建与版本管理时序图

内容状态机图占位（待补充）

图 3-11: 内容状态机图

（3）内容权限与审批发布。平台在空间与应用维度进行访问隔离，并在内容粒度通过 content_auth 实现策略式授权，以满足跨应用引用与精细化操作

控制需求。对于管理端操作，平台结合 RBAC 角色体系完成基础权限控制；对于内容对象，平台通过“授权对象 + 策略 + 变量 + 权限码”的模型表达细粒度权限。RBAC 权限矩阵如表 3-24所示。鉴权决策流程与审批发布链路分别如图 3-12与图 3-13所示。

表 3-24: RBAC 权限矩阵表

角色	空间/应用配置	创建/编辑	删除	提审	审批	上线/下线	查看/检索
系统管理员	✓	✓	✓	✓	✓	✓	✓
内容编辑者		✓		✓			✓
审核员					✓	✓	✓
普通用户							✓

内容权限鉴权流程图占位（待补充）

图 3-12: 内容权限鉴权流程图

在内容发布环节，平台严格遵循“编辑—提审—审批—发布”的流转机制。内容上线前必须经过指定审核员的审批，审批通过后系统生成发布版本，并通过事件机制异步通知索引构建、缓存刷新与数据统计等下游模块，确保发布过程的原子性与数据最终一致性。

（4）内容组织与应用引用。为提升内容可发现性与复用效果，平台提供类目管理、标签管理与应用引用内容机制。类目以树状层级构建稳定导航入口，标签以扁平维度实现横向聚合；应用引用内容用于表达应用可使用哪些已发布内容，使权威口径能够在多个应用复用并保持一致。应用引用内容的关键交互过程如图 3-14所示。

3.4.2 内容应用模块设计

内容应用模块面向内容消费场景提供统一的展示、检索与问答能力，对应需求分析中的” 内容应用功能需求” 部分。该模块以内容管理模块产出的” 已发

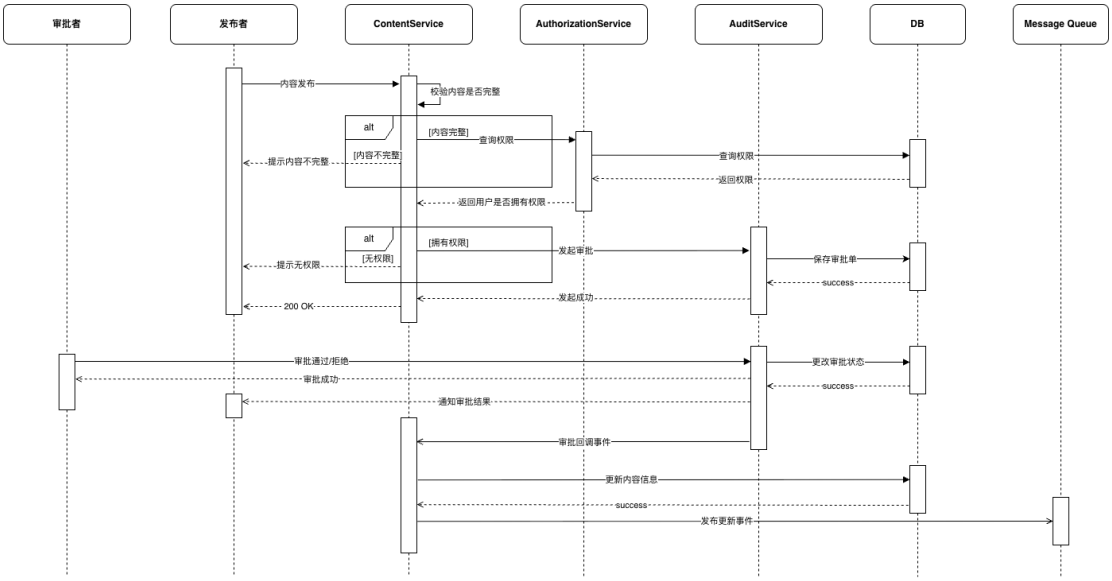

图 3-13: 内容发布流程时序图

应用引用内容时序图占位（待补充）

图 3-14: 应用引用内容时序图

布内容版本”作为输入，一方面为内容门户与管理后台提供页面化的内容展示与搜索能力；另一方面为上游 AI 助手提供生成式问答接口，实现”检索增强生成（RAG）+ 引用可追溯”的智能问答能力。以下对内容应用模块的主要功能子模块进行详细设计。

（1）内容展示。内容展示能力主要服务于内容门户与管理后台等页面化场景。平台基于空间与应用提供内容目录导航、列表聚合与详情页展示能力。目录导航以类目树为主干，配合标签体系提供横向聚合入口；列表聚合支持按时间、热度与业务规则进行排序；详情页展示需呈现内容来源、版本与关联内容，保证用户能够理解当前口径并快速跳转至相关内容。内容展示的主要交互过程如图3-15所示。

（2）内容检索。内容检索能力主要服务于关键词检索与结构化筛选。平台支持对标题、摘要、正文与结构化字段进行组合检索，并保证检索结果严格受空间、应用与 content_auth 的权限控制。检索链路由请求解析、范围确定、

内容展示流程/页面效果图占位（待补充）

图 3-15: 内容展示流程示意图

权限过滤、召回与排序、结果组装与高亮等步骤构成；其中范围确定负责将“空间/应用配置”与“调用方身份”映射为可搜索范围，权限过滤负责对召回候选进行二次过滤以避免越权结果暴露。内容检索的链路示意如图3-16所示。

内容搜索链路示意图占位（待补充）

图 3-16: 内容搜索链路示意图

（3）索引构建。索引构建是连接内容管理与内容应用的关键环节，其目标是在内容发布后将可用内容同步为面向检索与问答的索引结构。平台在发布事件驱动下触发索引构建流程，并针对不同内容形态执行一致的标准化处理，包括正文清洗、结构化字段抽取、元数据归一与切片生成等。对于页面检索场景，平台构建全文索引以支持关键词匹配与结构化过滤；对于生成式问答场景，平台基于切片生成向量索引以支持语义召回。索引构建流程如图3-17所示。

为保证索引与内容版本的一致性，索引构建过程需要以发布版本为输入，并在构建完成后以原子方式对外切换可用索引，避免出现检索结果与内容展示不一致的问题。

（4）生成式问答。为支撑智能问答场景，平台需要提供面向生成模型的可控内容支撑能力。该能力的关键在于将内容从“可读的文档”转化为“可检索的证据片段”，并在结果中返回可定位的来源信息。平台在索引构建阶段对不同内容形态进行前置处理，生成 document 与 slice 作为证据载体，并分别构建全文索引与向量索引以支撑多通道召回策略。

索引构建流程示意图占位（待补充）

图 3-17: 索引构建流程示意图

在问答阶段，平台采用多通道召回策略：一方面通过全文检索获取关键词高匹配证据，另一方面通过向量检索获取语义相近证据，并在融合阶段进行去重与重排，最终输出证据片段集合供生成模型组织回答。为便于后续治理与评估，问答链路需记录召回通道、得分、排序依据与引用来源，并在返回结果中携带内容标识与片段定位信息。生成式问答的召回与重排过程如图3-18所示。

RAG 召回与重排流程示意图占位（待补充）

图 3-18: 内容召回与重排流程示意图

3.4.3 内容治理模块设计

内容治理模块面向内容质量问题构建“发现问题—任务化闭环—复检回收”的治理体系，围绕重复度、歧义度与覆盖度三个方向，将质量问题从结果侧可观测化，并转化为可定位、可执行、可统计的治理工作，最终反哺内容资产与检索问答链路，形成持续演进机制。

（1）治理目标与度量。内容质量体系从内容覆盖、内容质量与效果评估三个方向建立指标口径，并结合客观质量与主观反馈实现质量度量。在内容覆盖方面，关注在给定阈值下用户原声是否能够召回到相关知识；在内容质量方面，关注内容重复与口径冲突带来的冗余与歧义；在效果评估方面，关注内容在检索与问答链路中的命中、引用与会话解决效果等指标，为治理优先级与策略调整提供依据。

（2）重复度/歧义度模型。重复度与歧义度检测当前覆盖 knowledge 与 faq 两类内容形态，其核心思路是采用“候选召回 + 大模型判别”的方式避免全量两两比较。首先对每个待检测内容在检索侧召回 topk 相似候选，形成候选对集合；随后对“当前内容—候选内容”构造判别提示词，使用大模型输出相似度与歧义度判断及可解释理由；最后将判别结果汇总为相似对、重复组与歧义候选组，并输出来源定位信息供人工复核与任务生成。重复度与歧义度检测流程如图 3-19所示。

重复度/歧义度模型检测流程示意图占位（待补充）

图 3-19: 重复度/歧义度模型检测流程示意图

（3）覆盖度治理机制。覆盖度治理用于持续发现“缺知识”与薄弱领域，并将缺口转化为可闭环的补充任务。其一，基于用户原声的覆盖治理以真实会话为驱动：在设定会话周期与召回阈值的前提下，对用户原声执行召回评估，召回结果未满足阈值要求的视为缺知识；对缺知识原声进行聚类，将相似原声聚为一类生成覆盖任务，由业务人员补充内容或完善现有内容，并通过每日定时巡检对任务范围进行复核，避免“内容已补充但任务未更新”导致统计失真。其二，基于内容先验的覆盖治理通过知识图谱 Diff 对比业务知识基座与内容中台知识库，发现缺失实体/关系与过时实体/关系等差异，形成可治理的问题清单并进入任务闭环。

（4）质量治理任务线上化。为将质量治理从离线跑数与人工推动升级为线上闭环，平台将治理过程抽象为主任务与子任务两层结构。主任务负责一次治理计算的配置与状态管理，包含任务名称、空间、会话周期与召回阈值等上下文信息；主任务进入计算中后生成治理任务列表，并在计算完成后对外展示结果。子任务主要包含覆盖任务与重复歧义任务两类：覆盖任务承载缺知识聚类结果，支持人工标记“无需补充/需要补充”并填写说明，系统每日巡检若发现已可召回到相关答案则自动完成；重复歧义任务承载重复对与歧义对结果，支持人工判断是否需要下线并完成处理标记，系统每日巡检若发现内容变更、下线或删除后不再存在重复/歧义，则自动完成。质量治理任务线上化的结构与关

键流程如图3-20所示。

质量治理任务线上化流程示意图占位（待补充）

图 3-20: 质量治理任务线上化流程示意图

（5）复检回收与链路一致性。治理任务处理通常会引发内容修订与再次发布，平台需要保证“内容资产—索引—问答结果”在治理后保持一致。对于需要修订的内容，治理处理应与内容管理的提审与发布流程打通，发布后由索引构建服务消费发布事件完成增量索引更新；随后对任务范围执行复检回收，确认质量问题是否消除，并将复检结果与指标变化回收至数据洞察体系，用于形成治理前后对比与策略迭代。跨域协同链路示意如图3-21所示。

治理修订-发布-复检时序图占位（待补充）

图 3-21: 治理修订-发布-复检时序图

3.4.4 数据洞察模块设计

数据洞察模块为平台提供效果评估与问题定位能力，是“应用驱动生产、数据反哺治理”的关键闭环环节。该模块围绕“采集—聚合—分析—下钻”四个层次构建：采集层记录内容曝光、点击、检索请求、召回结果、问答引用与用户反馈等数据；聚合层对原始数据进行清洗与指标化，形成面向空间、应用与内容维度的统计结果；分析层提供会话分析、效果评估与问题定位能力；下钻层支持从指标概览进入会话明细，查看具体问题、证据引用与返回结果，以便定位低命中或高转人工的原因。数据采集与指标生成链路示意如图3-22所示。

（1）数据采集。数据采集需要覆盖页面化消费链路与智能问答链路两类场景。在页面化链路中，平台采集曝光、点击、停留、搜索输入与结果点击等

数据采集与指标生成链路示意图占位（待补充）

图 3-22: 数据采集与指标生成链路示意图

行为数据；在问答链路中，平台采集问题输入、召回证据、引用内容、生成结果与用户反馈等链路数据。为避免采集口径不一致导致分析偏差，平台对埋点事件与字段进行统一定义，并在支撑服务层提供数据埋点服务对上游调用进行标准化封装。

（2）指标聚合与问题定位。在指标体系设计上，平台同时从内容维度与会话维度衡量效果。内容维度指标用于评估内容可发现性与贡献度，例如内容被检索命中次数、问答被引用次数等；会话维度指标用于评估链路效果与用户体验，例如命中率、转人工率与反馈信号等。平台在分析与下钻能力中支持从指标异常快速定位到问题会话与证据引用，形成“发现问题—定位原因—触发治理”的反馈通道。

数据洞察指标体系示意图占位（待补充）

图 3-23: 数据洞察指标体系示意图

3.5 数据库设计

平台的数据存储与检索体系需要同时满足内容管理的结构化写入、内容正文的灵活存储、检索与问答的高效召回，以及数据洞察的离线统计分析等需求。因此平台采用“关系型存储 + 文档存储 + 缓存 + 对象存储 + 检索索引 + 离线数仓”的组合方式，并通过消息队列与索引构建服务将内容发布与下游索引更新解耦。在关系型数据库中，平台主要存储空间、应用、内容元数据、版本、

授权策略、审批、索引元信息与治理任务等强一致数据；内容正文与形态扩展字段以引用方式关联文档存储或对象存储；离线洞察指标由数据平台按约定周期产出并供平台查询展示。

3.5.1 概念结构设计

图 3-24所示为平台的关键实体关系图。平台以空间与应用作为组织边界，其中空间用于定义业务归属与权限边界，应用用于定义内容消费场景与配置边界。在内容域中，平台以内容作为统一抽象实体，并通过内容类型字段区分四类内容形态（knowledge、faq、course、case），内容的具体版本通过内容版本实体进行管理。为支持跨应用复用，平台将“应用引用内容”抽象为应用-内容引用关系，用于表达某一应用可使用哪些内容。内容可见性与操作权限通过授权策略进行表达，授权策略在内容粒度上绑定主体与动作，以支撑空间/应用隔离下的精细化控制。发布上线依赖审批表记录一次提审的审批状态与审批意见，并在通过后驱动内容状态流转。为支撑 RAG 检索链路，平台以文档与切片描述索引对象：文档由多个切片组成，内容与文档通过关联表建立对应关系，从而将内容版本稳定映射到可检索证据片段。

数据库 E-R 图占位（待补充）

图 3-24: 数据库 E-R 图

3.5.2 表结构设计

本节对平台关键数据表进行说明。平台核心业务表统一采用自增主键 id，并在需要进行责任追溯与数据回收的表中保留 creator_id、create_time、modifier_id 与 modify_time 四个字段，以及 is_deleted 逻辑删除标识。对于内容正文与富文本等大字段，本系统在关系型数据库中仅保存引用标识（如对象存储键或文档库标识），避免大字段影响事务写入性能。

（1）空间与应用表。空间表用于存储空间的基础信息与负责人信息，

应用表用于存储应用的归属空间与基础配置。空间表与应用表结构分别如表3-25与表3-26所示。

表 3-25: 空间表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
name	空间名称	varchar(255)	空间的业务标识名称
owner_id	负责人 ID	bigint	空间负责人用户标识
status	状态	varchar(50)	ENABLED/DISABLED 等
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-26: 应用表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
space_id	空间 ID	bigint	外键，关联空间
name	应用名称	varchar(255)	应用的业务标识名称
type	应用类型	varchar(50)	门户/智能助手/开放接口等
status	状态	varchar(50)	ENABLED/DISABLED 等
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

（2）内容与版本表。内容表用于存储统一内容实体的元数据字段，通过 content_type 区分 knowledge、faq、course、case 四类内容形态；内容版本表用于记录内容的版本链与大字段引用。对于多形态内容的差异字段，平台采用“形态扩展表/详情存储”方式进行承载，其中 ext_json 用于保留少量通用扩展字段。内容表与内容版本表结构分别如表3-27与表3-28所示。

表 3-27: 内容表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
space_id	空间 ID	bigint	外键，关联空间
source_app_id	源应用 ID	bigint	内容归属的应用
content_type	内容类型	varchar(50)	knowledge/faq/course/case
title	标题	varchar(255)	
summary	摘要	varchar(1024)	
status	状态	varchar(50)	DRAFT/PENDING_REVIEW/PUBLISHED/OFF
current_version_id	当前版本 ID	bigint	关联内容最新版本
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-28: 内容版本表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	外键，关联内容
version_no	版本号	int(11)	递增版本号
change_summary	变更摘要	varchar(1024)	版本变更说明
body_ref	正文引用	varchar(255)	文档库标识或对象存储键
ext_json	扩展字段	text	形态扩展字段 JSON
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

（3）多形态内容扩展表。为体现 knowledge、faq、course、case 四类内容的差异化字段设计，平台在统一内容表的基础上，为各形态提供扩展表或详情存储。知识（knowledge）、FAQ（faq）与课程（course）的差异字段存储在关系型数据库的形态扩展表中，并与内容表通过 content_id 关联；课程由多个课节构成，课节信息通过 lesson 表进行维护。案例（case）的详情数据存储在 MongoDB 中，其中除少量基础字段外，其余字段统一存放于 JSON 详情字段，以适配案例结构的灵活演进。相关表结构如表 3-29至表 3-33所示。

表 3-29: 知识内容扩展表（knowledge_content）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	外键，关联内容表
editor_type	编辑器类型	varchar(50)	富文本/Markdown 等
keywords	关键词	varchar(255)	可选，用于运营标注
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-30: FAQ 内容扩展表（faq_content）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	外键，关联内容表
question	标准问题	varchar(1024)	FAQ 标准问法
answer_ref	答案引用	varchar(255)	答案正文引用标识
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-31: 课程内容扩展表（course_content）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	外键，关联内容表
cover_ref	封面引用	varchar(255)	封面图片对象存储键
level	难度等级	varchar(50)	初级/中级/高级等
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-32: 课节表（lesson）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
course_content_id	课程扩展 ID	bigint	外键，关联 course_content
title	课节标题	varchar(255)	
lesson_no	课节序号	int(11)	课程内排序序号
content_ref	课节内容引用	varchar(255)	课节内容对象存储键或文档引用
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

（4）应用引用与内容权限表。应用引用表用于表达应用可使用哪些内容，以支撑跨应用复用；内容权限表（content_auth）用于表达内容在可见性与操作权限层面的授权策略。内容权限表以“授权对象 + 策略 + 变量 + 权限码”的方式描述授权，其中 mapping_type 与 mapping_id 用于标识被授权的资源对象（如内容或应用等），auth_tactic 用于标识采用的策略类型，auth_variable 用于记录策略所需的变量值，auth_code 用于描述操作权限集合（例如 r/w/u 分别表示读、写、更新权限）。应用引用表与内容权限表结构分别如表 3-34与

表 3-33: 案例详情集合（MongoDB：case_detail）

字段名	字段中文名	字段类型	说明
_id	主键 ID	ObjectId	MongoDB 自带主键
content_id	内容 ID	bigint	关联统一内容表的 id
title	标题	string	基础字段，便于查询与展示
summary	摘要	string	基础字段，便于列表展示
detail	详情字段	json	案例的其余结构化/非结构化字段
creator_id	创建人 ID	long	
create_time	创建时间	datetime	
modifier_id	更新人 ID	long	
modify_time	更新时间	datetime	
is_deleted	是否删除	int	0 未删除，1 已删除

表 3-35所示。

表 3-34: 应用引用内容表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
app_id	应用 ID	bigint	外键，关联应用
content_id	内容 ID	bigint	外键，关联内容
ref_type	引用类型	varchar(50)	REFERENCE/ALIAS 等
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

（5）审批表。审批表用于记录一次内容提审的审批状态、审批意见与审批人信息。为保证审批对象明确，审批表关联目标内容版本 ID。审批表结构如表 3-36所示。

（6）索引文档与切片表。为支撑全文检索与向量召回，平台将可索引对象抽象为文档（document）与切片（slice）：一个文档可包含多个切片，切片用于向量化与证据定位；内容与文档通过关联表建立对应关系，以实现内容

表 3-35: 内容权限表（content_auth）

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
mapping_type	授权类型	tinyint	被授权对象类型标识
mapping_id	类型 ID	bigint	被授权对象的唯一标识
auth_tactic	权限策略	varchar(50)	策略类型（如员工匹配等）
auth_variable	权限变量	varchar(200)	策略变量值（如员工 ID、部门 ID 等）
auth_code	操作权限	varchar(20)	权限码集合（如 r/w/u）
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-36: 审批表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	外键，关联内容
target_version_id	目标版本 ID	bigint	外键，关联内容版本
submitter_id	提交人 ID	bigint	用户标识
reviewer_id	审批人 ID	bigint	用户标识，可为空
status	审批状态	varchar(50)	PENDING/APPROVED/REJECTED/CANCELLE
comment	审批意见	varchar(1024)	通过/拒绝原因说明
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

版本到检索证据的稳定映射。文档表、切片表与内容-文档关联表结构分别如表3-37、表3-38与表3-39所示。

表 3-37: 文档表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
status	状态	varchar(50)	ENABLED/DISABLED 等
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-38: 切片表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
document_id	文档 ID	bigint	外键，关联文档
slice_no	切片序号	int(11)	文档内序号
text_ref	文本引用	varchar(255)	切片文本引用或直接存储标识
offset_start	起始位置	int(11)	片段在文档内的起始偏移
offset_end	结束位置	int(11)	片段在文档内的结束偏移
meta_json	元数据	text	召回与可见性相关元信息
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

（7）治理任务表。治理任务用于承载重复、歧义与覆盖等治理问题的处理闭环，任务中包含问题定位、责任归属与处理状态等字段。治理任务表结构如表3-40所示。

表 3-39: 内容-文档关联表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
content_id	内容 ID	bigint	外键，关联内容
document_id	文档 ID	bigint	外键，关联文档
version_id	版本 ID	bigint	外键，关联内容版本
purpose	用途	varchar(50)	INDEX/RECALL 等
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

表 3-40: 治理任务表

字段名	字段中文名	字段类型	说明
id	主键 ID	bigint	自增主键
task_type	任务类型	varchar(50)	DUPLICATE/AMBIGUOUS/COVERAGE 等
priority	优先级	varchar(50)	P0/P1/P2 等
status	状态	varchar(50)	TODO/IN_PROGRESS/REVIEW/DONE/CLOSED
assignee_id	责任人 ID	bigint	用户标识
locator_json	问题定位	text	关联内容/切片与证据定位信息
result_json	处理结果	text	合并/修订/下线等结果说明
creator_id	创建人 ID	bigint	
create_time	创建时间	datetime	
modifier_id	更新人 ID	bigint	
modify_time	更新时间	datetime	
is_deleted	是否删除	tinyint	0 未删除，1 已删除

3.6 本章小结

本章围绕面向广告与商业化内容的管理、应用与治理平台展开分析与设计。首先从业务与系统视角给出了平台的整体定位与关键链路，明确平台由内容管理、内容应用、内容治理与数据洞察四个核心域构成，并以统一数据模型、权限体系与事件机制实现协同。随后在需求分析中按照模块给出了核心用例与用例描述表，明确平台在页面检索与智能问答场景下的能力边界，以及治理闭环与效果评估的关键需求。在总体设计与模块设计部分，本章给出了平台的分层架构与关键模块设计思路，并说明了内容发布后的事件驱动机制与索引构建策略。最后在数据库设计部分对平台的核心数据对象与存储选型进行了说明，为后续系统实现与效果评估奠定基础。

第四章　内容管理、应用与治理平台的实现

4.1　内容管理模块的实现

内容管理模块是平台的内容资产底座，承担内容生产、组织、权限与发布等职责，是后续内容应用与内容治理的唯一权威数据源。实现层面，模块围绕统一内容模型构建空间与应用组织、内容与版本管理、类目与标签组织、跨应用引用、权限控制与审批发布等能力，并通过事件机制在发布后驱动索引构建与数据回收，保证“发布版本”口径在展示、检索、问答与治理链路中的一致性。

参考师兄论文在实现章节中的组织方式，本节首先给出内容管理模块对外提供的关键接口概览，随后按子模块展开说明其核心实现机制。内容管理模块的关键服务端接口定义如表4-1所示。

表4-1中列出的接口覆盖了内容管理模块的三条主链路：组织边界建立（空间/应用）、内容生产与发布（内容/版本/审批）、以及复用与访问控制（组织/引用/权限）。创建空间与创建应用接口用于初始化业务隔离边界与消费边界，后续所有内容都需归属到空间与源应用中。内容创建、编辑与草稿保存接口用于支撑高频内容生产活动，并通过版本链承载变更历史。提审、审批与发布接口用于将内容从可编辑态切换到对外可用态，并为下游检索问答与治理模块提供“发布版本”的统一输入口径。类目树与标签接口用于组织内容并服务于导航与筛选。应用引用接口用于跨应用复用权威内容并沉淀影响范围。最后，权限相关接口为后续读链路的范围确定与结果过滤提供约束条件，是保证跨应用复用不导致越权暴露的关键。

表 4-1: 内容管理模块接口表

服务名称	接口名称	功能描述
空间/应用管理	createSpace / updateSpace / getSpaceDetail	创建与维护业务空间，作为 界。
空间/应用管理	createApp / updateApp / getAppDetail	在空间内创建应用并配置消 围。
内容管理	createContent / updateContent	创建与编辑内容元数据，写 本链。
内容管理	saveDraft / getDraft	保存草稿并获取可编辑版本 定控制。
内容管理	submitReview / approve / reject	发起提审、审批通过或驳回 并记录审计。
内容管理	publish / offline	发布生成对外可见的发布版 外不可见。
组织能力	upsertCategory / moveCategory / upsertTag	维护应用内类目树与标签体 与聚合展示。
引用能力	addReference / removeReference / getReferenceList	配置应用引用内容关系，支 范围追溯。
权限能力	checkPermission / getContentAuth	基于 RBAC 与 content_auth 查询。

4.1.1 多形态内容统一建模与管理

多形态内容统一建模是内容管理模块实现的基础。平台将 knowledge、faq、course、case 四类内容抽象为统一的内容实体 Content，并将内容的可编辑大字段与变更历史沉淀在内容版本 ContentVersion 中，从而实现“内容元数据稳定、正文与结构灵活演进、发布口径一致”的工程目标。其核心实现要点包括统一元数据字段设计、多形态差异字段承载、版本链维护以及面向发布的状态与版本约束。

类似师兄论文在 4.1 节中对“关键机制 + 关键实现片段”的组织方式，本节围绕“内容创建与编辑、版本链维护、状态流转约束”给出实现说明，并通过关键实现片段支撑论述。内容创建与版本管理的交互过程与状态流转可分别与图 3-10与图 3-11对应理解。

为便于后续内容应用、治理与洞察模块在实现上复用统一口径，内容管理模块在存储层形成了以空间、应用、内容、版本为主干的结构化表设计，并为

不同形态提供扩展表或详情存储。内容管理模块的核心数据表与职责划分如表4-2所示。

表 4-2: 内容管理模块核心数据表

数据对象	关键字段（示例）	说明
Space	id, name, owner_id, status	业务归属与隔离边界，承载成员与默认权限策略。
App	id, space_id, name, type, status	内容消费场景与配置边界，定义可用形态、组织方式与范围。
Content	id, space_id, source_app_id, content_type, title, summary, status, current_version_id	统一内容实体元数据，content_type 区分 knowledge/faq/course/case。
ContentVersion	id, content_id, version_no, change_summary, body(大字段), ext_json	内容版本链，承载可编辑正文与版本快照，用于草稿与发布口径。
knowledge_content	content_id, editor_type, keywords	知识类差异字段扩展，记录编辑器类型与运营标注。
faq_content	content_id, question, answer(大字段)	FAQ 差异字段扩展，以标准问答对为核心语义单元。
course_content	content_id, cover_ref, level	课程差异字段扩展，配合课节表表达学习路径。
lesson	course_content_id, title, lesson_no, content(大字段)	课程课节明细，lesson_no 用于确定课程内顺序。
case_detail(MongoDB)	content_id, title, summary, detail(json)	案例详情为灵活结构，采用文档存储承载可变字段。
AppContentRef	app_id, content_id	应用引用内容关系，用于跨应用复用与影响范围分析。
content_auth	mapping_type/id, auth_tactic, auth_variable, auth_code	内容粒度策略授权，与 RBAC 共同完成访问控制。

（1）统一内容实体与版本链维护。平台在关系型数据库中维护内容表与内容版本表：内容表记录空间、源应用、类型、标题、摘要、状态等元数据，

并以 current_version_id 指向最新草稿版本；内容版本表记录每一次编辑产生的版本节点，承载正文内容、结构化字段快照与版本号等信息。该设计使内容编辑过程可以在“草稿版本”上持续迭代，而对外生效的内容以“发布版本”为准，从数据层面约束了消费口径的一致性。

（2）内容创建与版本递增关键实现。内容创建与编辑的核心在于稳定维护版本链，并避免并发编辑导致的数据覆盖。创建时，系统在同一事务内写入 Content 元数据并初始化草稿版本；编辑保存时，系统生成新的版本节点并进行条件更新校验，确保写入基于最新版本。内容创建与草稿保存的关键实现片段如图4-1所示。

```
public Long createContent(CreateReq req) {
  // 1) 写入内容元数据
  Long contentId = contentDao.insertContent(req.spaceId, req.
      appId, req.type, req.title, req.summary);
  // 2) 初始化草稿版本（version_no = 1）
  Long versionId = versionDao.insertVersion(contentId, 1, req
      .body, req.changeSummary);
  // 3) 回写当前草稿版本
  contentDao.updateCurrentVersion(contentId, versionId);
  return contentId;
}

public Long saveDraft(SaveDraftReq req) {
  // 1) 获取当前版本号（由客户端携带，用于并发控制）
  int currentNo = versionDao.getVersionNo(req.contentId, req.
      baseVersionId);
  // 2) 生成新版本节点
  Long newVersionId = versionDao.insertVersion(req.contentId,
       currentNo + 1, req.body, req.changeSummary);
  // 3) 条件更新：仅当 current_version_id 未变化时才切换到新
      版本
  boolean ok = contentDao.compareAndSetCurrentVersion(req.
      contentId, req.baseVersionId, newVersionId);
  if (!ok) {
    throw new ConflictException("content version changed,
        please refresh");
  }
  return newVersionId;
}
```

图 4-1: 内容创建与版本管理关键代码

（2）多形态差异字段的具体落地。不同内容形态的差异不仅体现在“是否有正文”这一层面，更体现在内容结构、编辑方式与消费方式上，平台在实现中

对四类内容分别做了面向工程的字段拆分与存储选择。

对于 knowledge，平台支持富文本与 Markdown 等编辑器形式，正文以版本为单位存储，扩展表记录 editor_type 等编辑器特征，便于在渲染与切片阶段进行差异化处理；同时支持 keywords 等运营标注字段用于检索权重与筛选配置。对于 faq，平台以“标准问题 + 标准答案”作为最小知识单元，其中 question 字段用于稳定表达问法，答案内容作为大字段存储并与内容版本关联，以保证问答口径可随版本演进并支持回溯。对于 course，平台将课程元信息与课节拆分：course_content 负责承载封面、难度等元信息，lesson 表以 lesson_no 维护课节顺序并存储每一节的内容，从而在页面展示与学习路径导航中能够按序组织。对于 case，平台除少量基础字段外，将其余结构化/非结构化详情统一沉淀在 MongoDB 的 detail(json) 字段中，以适配案例字段随业务演进频繁变化的特点，并降低关系型表结构变更成本。

（3）正文与大字段存储方式。当前系统实现中，内容正文、FAQ 答案、课节内容等大字段直接存储在 MySQL 中，并随版本或课节记录一并落库。与“仅保存引用标识”的方案相比，直接存储的实现路径更简单，读写链路无需额外依赖外部正文存储；同时也带来数据库体量增长与大字段读写成本上升的问题。因此在实现上，平台通过“版本表承载正文、扩展表承载形态差异字段、文档存储承载 case 详情”的分层方式，尽量将高频查询的元数据与低频读取的大字段分离，并在列表与搜索等场景默认仅读取元数据与必要摘要，避免大字段在非必要场景被频繁加载。

（4）状态与发布口径约束。内容生命周期至少包含草稿、待审批、已发布、已下线等状态，平台在状态机约束下实现编辑、提审、审批、发布与下线等操作。实现中，写链路在保存草稿时仅更新草稿版本；发布动作在审批通过后生成发布版本并触发发布事件，确保内容应用与治理模块消费的都是已发布版本，避免“草稿可见”导致的口径漂移。相关状态流转与版本管理的交互过程可与前文系统设计中的内容状态机图、版本管理时序图对应理解。

（5）版本快照与回溯能力。版本链不仅用于草稿编辑与发布口径控制，也用于支撑质量治理与运营追溯。一方面，平台在每次编辑保存时生成新的版本节点并记录 change_summary 等摘要信息，便于运营人员理解版本差异并快速定位变更原因；另一方面，平台在治理任务处理引发内容修订时，以“修订—提审—审批—发布”的流程生成新的发布版本，使治理前后的口径能够被稳定对比，并为复检回收提供明确的版本边界。

（6）创建与编辑的写入路径。四类内容在“统一内容实体 + 差异扩展”的模型下具备一致的写入骨架：创建时先写入 Content 元数据并初始化草稿版本，再按 content_type 写入对应的扩展信息或详情数据；编辑时在更新元数据的同时生成新的版本节点，并将变更摘要与关键字段快照随版本记录沉淀，以保证后续发布、检索与治理能够准确对齐到同一版本。以 case 为例，平台在 MySQL 侧维护用于列表展示与检索的基础字段（如 title、summary），在 MongoDB 侧以 content_id 作为关联键存储 detail(json) 详情，并在 Mongo 文档中冗余 title 与 summary 字段以降低跨存储查询的复杂度，从而在结构灵活性与工程实现成本之间取得平衡。

在实现中，为降低并发编辑造成的覆盖风险，平台在保存草稿时采用“版本号递增 + 条件更新”的方式进行一致性控制：写入层在生成新版本节点时同时校验目标内容当前版本是否与请求携带版本一致，避免不同编辑者在旧版本基础上写入导致的丢失更新；若校验失败，则返回冲突提示并要求刷新后重试。该策略不依赖特定框架与中间件，能够在关系型数据库事务范围内完成一致性约束，是内容高频编辑场景下更易落地的工程方案。

4.1.2 内容组织与引用机制

内容组织与引用机制用于提升内容的可发现性与复用效率，降低重复维护成本与口径不一致风险。实现层面，平台以“应用内类目树 + 标签体系 + 应用引用内容关系”为核心组织方式：类目树提供稳定的导航入口，标签提供跨类目聚合与筛选能力，引用关系提供跨应用复用的工程化落地。

（1）类目树与标签体系实现。类目树以应用为边界维护，采用父子层级结构表达目录导航关系，并对节点移动、删除等操作进行约束以避免形成环或不合法层级。为提升查询效率，平台在类目查询场景提供“按应用获取完整树”和“按节点获取路径/子树”等接口，并在内容展示时通过类目挂载关系完成目录聚合。标签体系以扁平集合方式管理，支持标签的新增、禁用与与内容的绑定关系维护，用于在门户与后台提供基于标签的聚合与筛选能力。

（2）应用引用内容关系实现。应用引用内容用于表达“某应用可使用哪些已发布内容”，其本质是应用与内容的多对多关系。实现中，平台通过 AppContentRef 表维护引用关系：写入引用时校验被引用内容的发布状态与调用方权限，避免将未发布或无权限内容暴露到目标应用；读取内容列表与检索范围确定时，以引用关系作为范围约束之一，从而在应用侧形成可控的内容可

见集合。该机制既支持权威内容在多个应用复用，也为后续治理与影响范围分析提供可追溯的引用边界。

为保证“可复用”与“可控”的一致性，引用关系写入并非简单的多对多插入，而需要同时满足内容状态、授权策略与范围配置等约束。引用配置的交互过程与影响范围传播可与图3-14对应理解。引用关系写入的关键实现片段如图4-2所示，其核心校验点包括：被引用内容必须处于可对外消费的发布状态、调用方在目标空间/应用具备引用权限、以及在内容粒度策略上不会导致越权可见。

```
public void addReference(AddRefReq req) {
  // 1) 校验调用方在目标应用的引用权限（RBAC + content_auth）
  authService.checkPermission(req.operator, req.targetAppId,
     "CONTENT_REF_ADD");
  // 2) 校验内容状态：仅允许引用已发布内容
  Content c = contentDao.get(req.contentId);
  if (!c.isPublished()) {
    throw new BizException("only published content can be
       referenced");
  }
  // 3) 记录引用关系（幂等写入）
  refDao.upsert(req.targetAppId, req.contentId);
}
```

图 4-2: 应用引用内容关键代码

（3）影响范围与一致性保障。内容被多个应用引用后，任何修订与发布都可能影响多个消费入口。平台在引用关系上建立索引，并在发布事件中携带空间、应用与内容标识，便于下游基于引用关系快速定位影响范围并执行索引更新、缓存刷新与指标回收，从而实现“引用复用”与“发布一致性”的协同。

4.1.3 权限管理与审批流程

内容管理模块同时面向管理后台与对外消费链路提供能力，因此权限体系需要同时覆盖“后台可操作范围”与“内容可见/可操作范围”。平台采用“RBAC + content_auth”的组合实现：RBAC 用于在空间与应用维度进行角色授权与模块级/操作级控制，content_auth 用于在内容粒度表达更细的策略约束，以支撑跨应用引用、精细化可见范围以及合规要求。

（1）RBAC 权限控制实现。RBAC 以用户—角色—权限项映射为基础，角色在空间或应用维度生效，权限项覆盖空间/应用配置、内容创建编辑、删除、

提审、审批、上下线与查看检索等操作。实现中，接口层在处理请求时首先完成身份校验与 RBAC 权限校验，确保调用方具备对应操作能力，再进入后续对象级策略判断，避免越权调用对系统造成影响。

在平台管理实现中，RBAC 以“权限码 + 作用域绑定”的方式落地。权限码用于统一表达模块级与操作级权限项，例如空间管理、应用管理、成员管理、角色分配、内容写操作与审批发布等；作用域用于表达授权生效范围（全局/空间/应用）。工程上通过用户-角色绑定表记录 (scope_type, scope_id) 并在鉴权时解析请求中的 space_id 与 app_id 确定作用域，再依据角色-权限映射表判断是否具备目标 permission_code，从而实现跨空间隔离与空间内精细授权。为保证治理闭环可追溯，空间/应用配置变更、成员与角色分配、审批发布等关键操作统一记录审计日志。

（2）content_auth 策略授权实现。针对内容可见性与细粒度动作控制，平台以“授权对象 + 策略 + 变量 + 权限码”的模型表达策略：授权对象通过 mapping_type 与 mapping_id 指向空间、应用或内容等资源；策略通过 tactic 与 variable 描述可见范围或动作约束；权限码 code 表达具体权限项。鉴权决策流程在实现上以“策略加载—变量解析—动作匹配—结果裁决”为主线，并在读链路对召回候选进行二次过滤，保证检索与问答场景下不会返回越权结果。

（3）审批与发布流程实现。为保证内容对外可用前的准确性与合规性，平台将发布动作纳入流程化审批控制：提审接口会创建审批记录并将内容状态置为待审批；审批通过后生成发布版本并更新内容状态为已发布，同时产生内容发布事件驱动下游异步链路；审批驳回则回写拒绝原因并将内容退回可编辑状态。发布与下线接口会在保证幂等性的前提下触发事件，以驱动索引增量更新与统计回收，确保内容资产、索引与问答结果在发布前后保持一致。

审批与发布链路的实现重点在于两点：其一是以状态机约束操作合法性，避免非法状态流转；其二是发布后通过事件驱动触发下游异步处理，使“发布版本”在索引与数据侧尽快达成一致。审批与发布的关键过程可与图 3-13以及图 3-21对应理解，关键实现片段如图 4-3所示。

（4）审计与追溯。平台对关键操作（权限变更、提审、审批、发布、下线、引用配置等）记录审计日志，审计数据与内容版本链共同构成责任追溯依据；在出现口径争议或质量问题时，能够定位到具体版本、审批意见与操作者，从而支撑治理闭环中的问题定位与复检回收。

```
public void approve(ApproveReq req) {
  authService.checkPermission(req.auditor, req.spaceId, "
     CONTENT_APPROVE");
  Approval a = approvalDao.get(req.approvalId);
  if (!a.isPending()) {
    throw new BizException("invalid approval status");
  }
  // 1) 审批通过：更新审批记录与内容状态
  approvalDao.pass(req.approvalId, req.comment);
  contentDao.markPublished(a.getContentId());
  // 2) 触发发布事件，驱动索引与指标异步更新
  eventBus.publish(new ContentPublishedEvent(a.getContentId()
     , a.getSpaceId(), a.getAppId()));
}
```

图 4-3: 审批通过与发布事件触发关键代码

4.2 内容应用模块的实现

内容应用模块面向最终用户与业务运营，提供内容展示、内容搜索与生成式问答等能力。与内容管理模块不同，内容应用链路强调高并发读性能、低延迟与可解释性：其一，所有对外可见结果必须以“发布版本”为输入口径，避免草稿内容对外暴露；其二，检索与问答必须结合权限与范围配置做过滤，保证跨应用复用不引入越权风险；其三，生成式回答必须提供证据引用，满足可追溯与治理闭环要求。内容应用模块的用例范围如图 3-2所示。

参考师兄论文实现章节的组织方式，4.2 节按“链路机制 □ 关键实现片段 □ 与流程图对应”的结构展开：首先说明页面化内容应用的范围确定与读口径，其次介绍多形态内容前置处理与索引构建，再说明 RAG 多通道召回与重排机制，最后给出生成式问答链路的实现与引用可追溯策略。

4.2.1 页面化内容应用机制

4.2.2 多形态内容前置处理与索引构建

页面化内容应用机制用于支撑内容门户与管理后台的通用展示场景，包括目录、列表与详情页等典型页面形态。其实现目标是将“空间/应用边界、组织方式（类目/标签）、发布口径、权限过滤”在读链路中统一起来，从而在不同入口下保持一致的可见范围与展示效果。页面化内容应用的整体读链路如图 3-15所示。

（1）范围确定与请求规约。页面化展示请求通常携带 space_id、app_id、category_id、tag 等参数。服务端在接入层首先完成身份校验，并根据调用方来源（门户/后台）与应用配置确定内容范围：一方面，以 space_id 与 app_id 作为强边界过滤条件；另一方面，在应用启用“引用内容”时，将 AppContentRef 作为可见集合约束，从而限定仅展示被引用且已发布的内容。

（2）发布版本读取与一致性口径。展示链路统一以发布版本为准：列表页仅读取内容元数据与必要摘要；详情页在校验权限后读取发布版本正文。该策略保证内容管理、索引与问答链路对外一致，避免草稿内容在门户侧被提前可见。

（3）权限过滤与越权防护。页面化展示链路在返回结果前执行权限过滤：先做 RBAC 的模块/操作级校验，再按 content_auth 策略对具体内容进行对象级判断。对于列表页，服务端在查询阶段尽量将权限条件下推，以减少后续过滤成本；对于详情页，服务端在读取正文前进行强校验，避免越权内容被渲染或被前端缓存。

（4）关键实现片段。页面化展示的关键在于“范围确定 □ 权限过滤 □ 发布版本读取”的顺序约束。其核心实现片段如图 4-4所示。

```
public Page<ContentItemDTO> listContents(ListReq req) {
  // 1) 范围确定：space/app + 引用关系（可选）
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  // 2) 权限过滤：RBAC + content_auth
  authService.checkPermission(req.operator, req.appId, ”
      CONTENT_READ”);
  // 3) 查询发布口径：仅返回已发布内容的元数据与摘要
  return contentQueryDao.queryPublished(scope, req.categoryId
      , req.tags, req.page, req.size);
}

public ContentDetailDTO getDetail(DetailReq req) {
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  Content c = contentDao.get(req.contentId);
  authService.checkContentVisible(req.operator, scope, c.id);
  // 读取发布版本正文并渲染
  ContentVersion v = versionDao.getPublishedVersion(c.id);
  return renderService.render(v.body, c.meta);
}
```

图 4-4: 页面化内容展示关键代码

4.2.3 基于 RAG 的多通道召回与重排

4.2.4 生成式问答流程实现

为支持高效检索与生成式问答，平台在内容发布后对多形态内容进行前置处理，并构建全文索引与向量索引。该链路由内容发布事件驱动：内容管理模块在审批通过后触发 ContentPublished 事件，索引构建服务消费事件并完成文档化、切片与索引写入，整体过程如图3-17所示。

（1）多形态前置处理策略。不同内容形态在索引侧的目标是统一为可检索的 Document 与最小证据单元 Slice。知识类与 FAQ 以正文段落或问答对为主要切片粒度；课程以课节为基本单元，并在每个课节内按段落切分；案例则对 detail(json) 中的关键字段做字段抽取与拼接，同时保留 title 与 summary 以增强召回稳定性。前置处理阶段会对 HTML 标签、富文本标记、无意义噪声进行清洗，并保留必要的结构信息（如标题层级）以便后续生成回答时组织引用。

（2）Document/Slice 构造与元信息绑定。索引对象需与内容资产口径对齐。实现中，Document 记录 content_id、version_id、space_id、app_id 等元信息，并维护索引更新时间与来源；Slice 记录 document_id、slice_id、text 等字段，并可选维护 embedding 向量。该绑定方式使任意召回结果都能回溯到具体内容与版本，为引用可追溯与治理定位提供基础。

（3）增量更新与幂等控制。索引构建消费发布事件后进行增量更新：同一 content_id 的旧版本索引会被标记为过期或被覆盖更新，保证查询侧只命中最新发布版本。为避免重复消费导致的脏写，索引构建服务以 (version_id, index_type) 作为幂等键，并在写入前检查已处理版本，从而保证“至少一次投递”语义下的最终一致。

（4）关键实现片段。索引构建核心在于“事件消费 □ 拉取发布版本 □ 切片 □ 双索引写入”。其关键实现片段如图4-5所示。

4.3 内容治理模块的实现

基于 RAG 的多通道召回与重排用于在生成式问答场景下提供高质量证据集合，并在可控成本下兼顾召回率与精度。实现层面，平台采用“多通道召回—融合去重—重排—证据组织”的流水线结构：全文通道负责精准关键词召回，向量通道负责语义相似召回，两者融合后进行去重与重排，最终输出与问题最相

```
public void onContentPublished(ContentPublishedEvent e) {
  // 1) 幂等：已处理版本直接返回
  if (indexMetaDao.exists(e.versionId, "FULLTEXT")) return;
  // 2) 拉取发布版本正文
  ContentVersion v = versionDao.getById(e.versionId);
  // 3) 前置处理与切片
  List<Slice> slices = sliceService.buildSlices(e.contentId,
     e.versionId, v.body, v.extJson);
  // 4) 写入全文索引与向量索引
  esClient.upsertDocuments(e.spaceId, e.appId, e.contentId, e
     .versionId, slices);
  vectorClient.upsertEmbeddings(e.spaceId, e.appId, e.
     contentId, e.versionId, slices);
  // 5) 记录索引元信息
  indexMetaDao.markDone(e.versionId);
}
```

图 4-5: 索引构建关键代码

关的 Slice 集合。整体过程如图 3-18所示。

（1）多通道召回。全文召回基于 ES 的 BM25 与字段权重，支持标题、摘要与正文多字段匹配，并结合空间/应用范围与权限条件进行过滤；向量召回基于 embedding 相似度检索，能够覆盖同义改写与长尾问法。为控制延迟，系统对两路召回设置 topK 上限，并在召回阶段尽量下推过滤条件，减少后续重排候选规模。

（2）融合去重与候选裁剪。多通道返回的候选以 slice_id 为主键进行融合去重，并对同一 content_id 的多个候选进行聚合，避免证据集中在单一内容上导致回答偏置。融合阶段会根据召回来源与初排分数进行粗排裁剪，为后续重排降低成本。

（3）重排与证据组织。重排阶段以多特征进行排序，包括语义相似分、关键词覆盖、标题命中、内容新鲜度等，并对候选进行多样性约束（例如限制同一文档的 slice 数量）。证据组织阶段将 topN 的 slice 按文档聚合，并截断拼接为适合大模型上下文窗口的证据段落，同时保留每段证据的来源标识，用于输出可追溯引用。

（4）关键实现片段。RAG 召回与重排的关键实现片段如图 4-6所示。

```
public EvidencePack retrieveEvidence(AskReq req) {
  Scope scope = scopeService.resolve(req.spaceId, req.appId,
      req.operator);
  authService.checkPermission(req.operator, req.appId, "
      QA_READ");

  List<SliceHit> esHits = esClient.search(req.query, scope,
      req.topkEs);
  List<SliceHit> vecHits = vectorClient.search(req.query,
      scope, req.topkVec);

  List<SliceHit> merged = mergeService.dedupAndMerge(esHits,
      vecHits);
  List<SliceHit> reranked = rerankService.rank(req.query,
      merged, req.topn);

  return evidenceBuilder.build(reranked); // 组织引用信息与上
      下文片段
}
```

图 4-6: RAG 召回与重排关键代码

4.3.1 重复度、歧义度与覆盖度检测

4.3.2 治理任务线上化与闭环机制

生成式问答流程将 RAG 检索得到的证据集合注入大模型提示词中生成回答，并在输出侧附带可追溯引用，形成“问答—引用—治理”的闭环入口。实现目标是：在保证回答准确性的前提下，降低幻觉风险，并通过引用定位到具体内容片段以支撑后续治理。问答流程实现可复用图3-18中“证据组织—生成回答—返回引用”的后半段链路。

（1）Prompt 构造与上下文控制。系统在构造提示词时将用户问题、业务约束（如回答风格、禁止编造）、以及证据集合分段拼接，并对证据长度进行裁剪以适配上下文窗口。为保证引用可追溯，系统在证据段落中显式附带 slice_id 与标题信息，使模型输出的引用能够映射回具体 Slice。

（2）回答生成与引用解析。模型生成结果返回后，系统对回答文本中的引用标识进行解析，并转换为可展示的“内容标题 + 片段摘要 + 跳转链接”。当模型未给出引用或引用解析失败时，系统退化为按证据集合的 topN 进行引用补齐，保证每次回答都有最小可追溯集合。

（3）异常退化与安全控制。当检索阶段未召回到足够证据时，系统返回

“未命中/建议补充内容”的提示，并将该问题作为覆盖度治理的候选输入；当模型生成超时或失败时，系统退化为检索结果摘要返回，保证服务可用性。读链路全过程保留权限过滤与范围约束，避免越权证据进入提示词。

（4）关键实现片段。生成式问答的关键实现片段如图4-7所示。

```
public AskResp ask(AskReq req) {
  EvidencePack pack = retrieveEvidence(req);
  if (pack.isEmpty()) {
    coverService.recordMiss(req); // 覆盖度治理候选
    return AskResp.miss("未命中可用内容，建议补充知识");
  }
  String prompt = promptBuilder.build(req.query, pack);
  String answer = llmClient.generate(prompt);
  List<Citation> cites = citationParser.parseOrFallback(
     answer, pack);
  return AskResp.ok(answer, cites);
}
```

图 4-7: 生成式问答关键代码

4.4 内容治理模块的实现

4.4.1 重复度、歧义度与覆盖度检测

4.4.2 治理任务线上化与闭环机制

4.5 数据洞察模块的实现

4.5.1 内容应用数据采集机制

4.5.2 会话分析与效果指标设计

4.6 本章小结

第五章　内容管理、应用与治理平台测试

5.1　系统测试环境

5.2　单元测试

5.3　功能测试

5.4　本章小结

第六章　总结与展望

6.1　总结

6.2　展望

参考文献

附录 A　附录示例

A.1　main 函数

```
int main()
{
        return 0;
}
```

致　　谢

致谢示例

版权及论文原创性说明

任何收存和保管本论文的单位和个人，未经作者本人授权，不得将本论文转借他人并复印、抄录、拍照或以任何方式传播，否则，引起有碍作者著作权益的问题，将可能承担法律责任。

本人郑重声明：所呈交的学位论文，是本人在导师的指导下，独立进行研究工作所取得的成果。除文中已经注明引用的内容外，本论文不包含其他个人或集体已经发表或撰写的作品成果。本文所引用的重要文献，均已在文中以明确方式标明。本声明的法律结果由本人承担。

作者签名：_____

_____年_____月_____日

《学位论文出版授权书》

本人完全同意《中国优秀博硕士学位论文全文数据库出版章程》（以下简称“章程”），愿意将本人的学位论文提交“中国学术期刊（光盘版）电子杂志社”在《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》中全文发表。《中国博士学位论文全文数据库》、《中国优秀硕士学位论文全文数据库》可以以电子、网络及其他数字媒体形式公开出版，并同意编入《中国知识资源总库》，在《中国博硕士学位论文评价数据库》中使用和在互联网上传播，同意按“章程”规定享受相关权益。

作者签名：_____

_____年_____月_____日

论文题名	面向广告与商业化内容的管理、应用与治理平台的设计与实现				
研究生学号	522024320076	所在院系	软件学院	学位年度	2026
论文级别	☐ 学术学位硕士　☐ 专业学位硕士 ☐ 学术学位博士　☐ 专业学位博士 （请在方框内画钩）				
作者 Email	522024320076@smail.nju.edu.cn				
导师姓名	刘峰 高级工程师				

论文涉密情况：

☐ 不保密

☐ 保密，保密期 (_____年_____月_____日 至 _____年_____月_____日)

注：请将该授权书填写后装订在学位论文最后一页（南大封面）。